

Noida Institute of Engineering & Technology, Greater Noida



(An Autonomus Institute)

LAB FILE

Branch: Mtech(Int.)-CSE

Semester: Vth

Subject Name: Advance Java Programming

Subject Code:BMICSE0556

Student Name: Pratigya

Faculty Name: Ms. Aditee Mattoo

Roll no: 2401330100536

**Department of Computer Science & Engineering
NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY**

**19, KNOWLEDGE PARK-II, INSTITUTIONAL AREA
GREATER NOIDA, (U. P.) - 201 306, INDIA**

INDEX

Program no.	Program Name	Date	Signature	Remark/ Grade
1	Install a database (MySQL or Oracle). Program to illustrate JDBC connectivity. Program for maintaining databases by sending queries.			
2	Write program to create a demo table emp having fields id, name, city and insert two rows by using JDBC.			
3	Write a Java program using Statement to create a table Students with fields id, name, age, and grade.			
4	Write a Java program using Statement to insert multiple records into the Students table			
5	Write a Java program to demonstrate a money transfer transaction between two bank accounts. Ensure that both debit and credit operations are either committed together or rolled back in case of an error.			
6	Write a Java program using Prepared Statement to delete student records where grade is below a given threshold.			
7	Write a Java program using PreparedStatement to insert multiple records efficiently using batch execution.			
8	Write a Java program to call a stored procedure that calculates the total salary of employees in a given department.			
9	Implement a java program using TCL statements commit (), rollback(), setAutoCommit(), setSavepoint(), and releaseSavepoint() method on Employee table.			
10	Write a servlet program to select the details of an employee (emp id, empname, empadd ,empphone) and display on browser in appropriate format.			
11	Write a GenericServlet to handle employee registration. The servlet should: Accept employee details (name, email, designation, salary) via a form (POST request).Store the details in a database (use JDBC). Display a success message after successful insertion.			
12	Implement a servlet program that receive two inputs Name and Password from HTML page			

	and display on the browser.			
13	Implement a servlet program that redirect a request to Google.com.			
14	Implement session handling concept by using HttpSession object.			
15	Implement session handling concept by using URL rewriting method.			
16	Create a servlet Program which displays cookie id with the help of session handling concepts.			
17	Implement a program that takes three input as: User Name, User Password and User Mobile from html form and access these data by using servlet also display these details on browser.			
18	Implement a servlet program to select the details of an employee (emp id, empname, empadd, empphone) and display on browser in appropriate format.			
19	Implement a program to add any data on cookies and also access these data from cookies.			
20	Implement servlet a program to implement the redirection of any request to other resources such as an html file.			
21	Create a table which should contain at least the following fields: name, password, email-id, phone number Implement a java program/servlet/JSP to connect to that database and extract data from the tables and display them. Insert the details of the users who register with the web site, whenever a new user clicks the submit button in the registration page.			
22	Design and implement a simple servlet book query such as book_id, book_name, book_author and published date with the help of JDBC & SQL. Create on ODBC/latest driver link, Compile & Execute JAVA JDBC Socket.			
23	Design a simple application program using Servlet and Database 1. Simple login form 2. Customer Feedback Form 3. Admission Form 4. Student Mark Sheet			

24	Assume four users user1, user2, user3 and user4 having the passwords pwd1, pwd2, pwd3 and pwd4 respectively. Implement a servlet for doing the following. Create a Cookie and add these four-user id's and passwords to this Cookie.2. Read the user id and passwords entered in the Login form and authenticate with the values available in the cookies.			
25	Implement all JSP scripting element (scriptlet, expression and declaration).			
26	Implement all JSP directive element (page, include and taglib).			
27	Implement JSP program that calculates factorial values for an integer number, while the input is taken from an HTML form			
28	Implement a JSP program for displaying basic arithmetic functions that calculates Powers of 2 for integers in the range 0-10.			
29	Implement a JSP (EmpBeanTest.jsp) page that illustrates how to access a JavaBean class by using all required action tags from a JSP page.			
30	Implement a JSP program to display strings that shows a sample order form in restaurant.			
31	Implement a JSP program to upload file into server.			
32	Implement a JSP program to count no. of visitors once user clicks on submit button on webpage.			
33	Implement a JSP program to illustrate session tracking for printing the string array (color) through index.jsp page and print the length of the selected color through another page print.jsp.			
34	Implement a JSP program for arithmetic exception error handling by using html page, jsp page, and errorPage, isErrorPage directives.			
35	Implement a JSP program to display current date & time of your system.			
36	Implement a JSP page to retrieve data from HTML into JSP and display information on browser.			
	A company's recruitment portal requires candidates to register before applying for jobs.			

37	Design a JSP page for registration and a Servlet to store user details in a MySQL database using JDBC. How would you implement session management in a JSP-based web application to keep users logged in across multiple pages?			
38	Implement a JSP program to validate username and password in sample order form.			
39	Implement a JSP program to select record from database Employee.			
40	Implement a JSP which insert the details of the 3 or 4 users who register with the web site by using registration form. Authenticate the user when he submits the login form using the user's name and password from the database. Design and implement a simple shopping cart example with session tracking API.			
41	Program: Create a simple Spring project using Maven that prints a welcome message using a Spring Bean. Concept: Introduction to Spring Ecosystem and Bean creation.			
42	Program: Demonstrate the use of ApplicationContext as IOC container to load beans. Concept: Spring IOC container usage.			
43	Program: Define multiple Spring Beans and manage them using XML-based configuration. Concept: Spring Modules and IOC in action.			
44	Program: Demonstrate Constructor-based DI in Spring. Concept: Injecting dependencies using constructor.			
45	Program: Implement Setter-based DI to inject a service bean into another. Concept: Using setters for bean wiring.			
46	Program: Demonstrate Field Injection using @Autowired annotation. Concept: Simplified DI using annotations.			
47	Program: Configure beans using Java-based configuration with @Configuration and @Bean. Concept: Pure Java Spring configuration.			

48	Program: Use Component Scanning and @Component, @Service, @Repository. Concept: Auto-detection of components.			
49	Program: Demonstrate Annotation-based configuration with full @Configuration and @ComponentScan. Concept: Clean and scalable configuration.			
50	Program: Create a simple JDBC DAO using Spring JDBC Template to fetch data from H2 database. Concept: Spring JDBC template integration.			
51	Program: Implement Insert and Update operations using Spring JDBC. Concept: Data manipulation using Spring.			
52	Program: Handle exceptions and use Row Mapper with Spring JDBC for object mapping. Concept: Robust data access layer.			
53	Program: Set up a Maven project structure for a Spring app and manage dependencies using pom.xml. Concept: Maven-based project setup.			
54	Program: Integrate external dependency (like MySQL connector) and test project compilation. Concept: Dependency management in Maven.			
55	Program: Create a modular app with User Service, injected via constructor, and use annotations (@Component, @Autowired). Concept: Real-world DI and service layer architecture.			
56	Program: Develop a full Spring app managing user profiles with Component Scanning, Java-based Config, JDBC for persistence, tested via H2 DB. Concept: Case Study combining all concepts into one application.			
57	Case Study: Develop a Spring application utilizing Dependency Injection, component scanning, and annotation-based configuration within a Maven-based setup			
	You are designing a web application for an			

58	online bookstore. Set up a basic Spring MVC structure with Dispatcher Servlet, controller, and view." Program Goal: Create a minimal Spring MVC app showing folder structure and web.xml setup.			
59	Build a feedback page for a college portal. Create a Spring MVC controller that returns a welcome message on accessing /feedback." Program Goal: Implement a basic controller class using @Controller and map a simple GET request.			
60	In a product catalog system, pass a product's name and price from the controller to the view for display." Program Goal: Use Model or ModelMap in the controller to send data to JSP.			
61	"Create a login form where the user inputs their username. Display a welcome message using data passed from the form." Program Goal: Use @RequestParam to capture form input and display it in JSP. any key.			
62	Build a registration form for students and bind form data to a Student object." Program Goal: Use @ModelAttribute to map form fields to a model bean.			
63	In a tourism site, display a list of popular destinations in a JSP page returned from a controller." Program Goal: Configure JSP view resolution in spring-servlet.xml and use JSTL to display dynamic content.			
64	Create a feedback form where name and message fields are mandatory. Validate the form input and show error messages." Program Goal: Use Spring Form tag library and BindingResult to perform simple validation.			
65	Design a student records system that fetches student data from a MySQL database and displays it using Spring MVC." Program Goal: Connect Spring MVC to a database using Spring JDBC and show data in a JSP.			
	Case: "Your team is building a microservice for user registration. Initialize a basic Spring Boot			

66	application structure." Program Goal: Use @SpringBootApplication, run the app, and understand the auto- configured structure.			
67	Case: "You want to quickly start a RESTful service for a contact book using Spring Initializr." Program Goal: Generate a Spring Boot project with Spring Web and Spring Boot DevTools.			
68	Case: "In a university portal, configure a controller without any XML and rely on auto-configuration." Program Goal: Use @RestController, @RequestMapping, and rely on application.properties.			
69	Case: "Build a student management module that stores data in an in-memory H2 database." Program Goal: Integrate Spring Data JPA with H2, create an entity, repository, and test data storage.			
70	Case: "Your library website needs to show a homepage with contact info and a static banner." Program Goal: Serve HTML from /templates and static images/CSS from /static.			
71	Case: "Create a contact form submission endpoint. Use GET to load the form and POST to save data." Program Goal: Implement separate methods for @GetMapping and @PostMapping.			
72	Case: "You want to reduce boilerplate in your Book class (getters, setters, constructors)." Program Goal: Use @Data, @NoArgsConstructor, @AllArgsConstructor from Lombok.			
73				

	Case: "Develop a course catalog system that allows adding, listing, updating, and deleting courses." Program Goal: Build full CRUD using Spring Boot, Spring Data JPA, H2, and REST endpoints.			
74	You are designing a simple library system. Begin by creating an entity Book and use JPA to persist data to a relational database.			
75	In a school database system, annotate an @Entity class Student with appropriate JPA annotations like @Id, @GeneratedValue, @Column.			
76	In a hospital management system, create a one-to-one relationship between Patient and Medical Record using JPA			
77	Implement CRUD operations on MYSQL database using spring data rest with POSTMAN client.			
78	Implement CRUD operations on POSTGRESQL database using spring data rest with POSTMAN client.			
79	Develop a system for online courses where one Instructor can have many Courses. Model this using JPA annotations.			
80	Build a college management system where Students can enroll in multiple Courses. Implement many-to-many using JPA.			
81	In a retail system, expose CRUD operations for Product entity using Spring Boot REST API.			
82	Develop an employee management system that performs Create, Read, Update, and Delete operations using JpaRepository			
83	In a book store app, compare the use of CrudRepository vs. JpaRepository for managing Book entities			
84	In a weather app, build an API that serves both JSON and XML responses using appropriate Spring annotations			
85	After building a student API, demonstrate how to test CRUD endpoints using Postman			

	including headers and body format			
86	Once your API is ready, deploy the project on localhost using Spring Boot and test endpoints in a browser or Postman			
87	Simulate a simplified Netflix system: design entities like User, Content, and Watch History using JPA relationships, and expose REST APIs for user registration and content tracking			
88	Case Study: Build A Netflix-like system to manage users, content, and viewing behavior using JPA.			

Program 1

Objective: Install a database (MySQL or Oracle).

Program Code

SQL Source File: Program1Source.sql

```
-- MySQL verification

CREATE DATABASE labdb;

USE labdb;

SELECT VERSION();
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

</dependencies>

</project>
```

Java File: JdbcConnectivity.java

```
import java.sql.*;

public class JdbcConnectivity {

    public static void main(String[] args) throws Exception {

        Class.forName("com.mysql.cj.jdbc.Driver");

        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/labdb", "root",
"root");

            Statement st = con.createStatement()) {

            ResultSet rs = st.executeQuery("SELECT VERSION()");

            if (rs.next()) System.out.println("MySQL version: " + rs.getString(1));

        }

    }

}
```

```
}
```

```
}
```

SQL File: labdb.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
SELECT VERSION();
```

Output



```
mysql> mysql -u root -p
Enter password:
mysql> CREATE DATABASE IF NOT EXISTS labdb;
mysql> USE labdb;
mysql> SELECT VERSION();
+-----+
| version() |
+-----+
| 5.7.32    |
+-----+
1 row in set (0.00 sec)

mysql>
```

Program 2

Objective: Create demo table emp and insert two rows using JDBC.

Program Code

Java Source File: Program2Source.java

```
import java.sql.*;

public class EmpJDBC {

    public static void main(String[] args) throws Exception {

        String url="jdbc:mysql://localhost:3306/labdb";

        try (Connection con=DriverManager.getConnection(url,"root","root");

            Statement st=con.createStatement()) {

            st.executeUpdate("CREATE TABLE IF NOT EXISTS emp("+
                "id INT PRIMARY KEY, name VARCHAR(50), city VARCHAR(50))");

            st.executeUpdate("INSERT IGNORE INTO emp VALUES"+
                "(1,'Rahul','Delhi'),(2,'Priya','Noida')");

            System.out.println("Table created and two rows inserted.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

    </dependencies>

</project>
```

SQL File: emp.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS emp;

CREATE TABLE emp (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  city VARCHAR(50)
);

INSERT INTO emp VALUES (1,'Aman','Noida'),(2,'Neha','Delhi');
```

Output



Program 3

Objective: Create Students table using Statement.

Program Code

Java Source File: Program3Source.java

```
import java.sql.*;

public class CreateStudents {

    public static void main(String[] args) throws Exception {

        try(Connection con=DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/labdb","root","root");

            Statement st=con.createStatement()) {

            st.executeUpdate("CREATE TABLE IF NOT EXISTS Students("+

                "id INT PRIMARY KEY, name VARCHAR(50), age INT, grade VARCHAR(5))");

            System.out.println("Students table created.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

    </dependencies>

</project>
```

SQL File: students.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS students;

CREATE TABLE students (

id INT PRIMARY KEY,

name VARCHAR(50),

age INT,

grade VARCHAR(5)

);
```

Output



Program 4

Objective: Insert multiple records into Students using Statement.

Program Code

Java Source File: Program4Source.java

```
import java.sql.*;

public class InsertStudents {

    public static void main(String[] args) throws Exception {

        try(Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root");
            Statement st=con.createStatement()) {

            st.executeUpdate("INSERT IGNORE INTO Students VALUES"+
                "(1, 'Aman', 20, 'A'), (2, 'Neha', 21, 'B'), (3, 'Ravi', 20, 'A')");

            System.out.println("Multiple student records inserted.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

    </dependencies>

</project>
```

SQL File: students_data.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
INSERT INTO students VALUES (1,'Aman',20,'A'),(2,'Neha',21,'B'),(3,'Ravi',19,'C');
```

Output



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\quptax\java\Program
Multiple student records inserted.
1 Aman 20 A
2 Neha 21 B
3 Ravi 20 A
```

Program 5

Objective: Demonstrate money transfer transaction between two bank accounts.

Program Code

Java Source File: Program5Source.java

```
import java.sql.*;

public class BankTransfer {

    public static void main(String[] args) throws Exception {

        try(Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root")) {

            con.setAutoCommit(false);

            try(PreparedStatement debit=con.prepareStatement(
                "UPDATE account SET balance=balance-? WHERE id=?");

                PreparedStatement credit=con.prepareStatement(
                    "UPDATE account SET balance=balance+? WHERE id=?")) {

                debit.setDouble(1,1000); debit.setInt(2,1); debit.executeUpdate();

                credit.setDouble(1,1000); credit.setInt(2,2); credit.executeUpdate();

                con.commit();

                System.out.println("Transfer committed successfully.");

            } catch(Exception e) {

                con.rollback();

                System.out.println("Error: transaction rolled back.");

            }

        }

    }

}
```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

</dependencies>

</project>

```

SQL File: bank.sql

```

CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS account;

CREATE TABLE account (

id INT PRIMARY KEY,

name VARCHAR(50),

balance DECIMAL(12,2)

);

INSERT INTO account VALUES (1, 'Aman', 5000), (2, 'Ravi', 3000);

```

Output

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gupta>java Program5
Transfer amount: ₹1000
Debit: Account 1
Credit: Account 2
Transaction committed successfully.

```

Program 6

Objective: Delete student records below a given grade threshold using PreparedStatement.

Program Code

Java Source File: Program6Source.java

```
import java.sql.*;

public class DeleteByGrade {

    public static void main(String[] args) throws Exception {

        String sql="DELETE FROM Students WHERE grade=?";

        try(Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root");

            PreparedStatement ps=con.prepareStatement(sql)) {

            ps.setString(1,"C");

            int n=ps.executeUpdate();

            System.out.println(n+" record(s) deleted.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

</dependencies>
```

</project>

SQL File: students_grade.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
-- Uses the Students table from Program 3.
```

```
SELECT * FROM students;
```

Output



Program 7

Objective: Insert multiple records efficiently using Prepared Statement batch execution.

Program Code

Java Source File: Program7Source.java

```
import java.sql.*;

public class BatchInsert {

    public static void main(String[] args) throws Exception {

        String sql="INSERT INTO Students(id,name,age,grade) VALUES(?,?,?,?)";

        try(Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root");
            PreparedStatement ps=con.prepareStatement(sql)) {

            int[][] data={{11,20},{12,21},{13,20}};

            String[] names={"Karan","Simran","Vikas"};

            for(int i=0;i<data.length;i++){

                ps.setInt(1,data[i][0]); ps.setString(2,names[i]);

                ps.setInt(3,data[i][1]); ps.setString(4,"A");

                ps.addBatch();

            }

            ps.executeBatch();

            System.out.println("Batch inserted successfully.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.compiler.target></properties>
```

```
<dependencies>
```

```
<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><version>9.3.0</version></dependency>
```

```
</dependencies>
```

```
</project>
```

SQL File: batch_data.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
-- Uses the Students table from Program 3.
```

```
SELECT * FROM students;
```

Output



Program 8

Objective: Call a stored procedure that calculates total salary for a department.

Program Code

Java Source File: Program8Source.java

```
-- MySQL

DELIMITER //

CREATE PROCEDURE dept_total(IN d VARCHAR(30), OUT total DECIMAL(12,2))

BEGIN

SELECT COALESCE(SUM(salary),0) INTO total

FROM employee WHERE department=d;

END//

DELIMITER ;
```

```
-- Java

import java.sql.*;

public class CallProcedure {

public static void main(String[] args) throws Exception {

try(Connection c=DriverManager.getConnection(

"jdbc:mysql://localhost:3306/labdb","root","root");

CallableStatement cs=c.prepareCall("{call dept_total(?,?)}")) {

cs.setString(1,"IT");

cs.registerOutParameter(2,Types.DECIMAL);

cs.execute();

System.out.println("IT Department Total = ₹"+cs.getBigDecimal(2));

}

}

}
```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

</dependencies>

</project>

```

SQL File: salary_procedure.sql

```

CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS employee;

CREATE TABLE employee (
emp_id INT PRIMARY KEY,
name VARCHAR(50),
department VARCHAR(50),
salary DECIMAL(12,2)
);

INSERT INTO employee VALUES (1,'Aman','IT',50000),(2,'Neha','HR',45000),(3,'Ravi','IT',60000);

DROP PROCEDURE IF EXISTS total_salary;

DELIMITER //

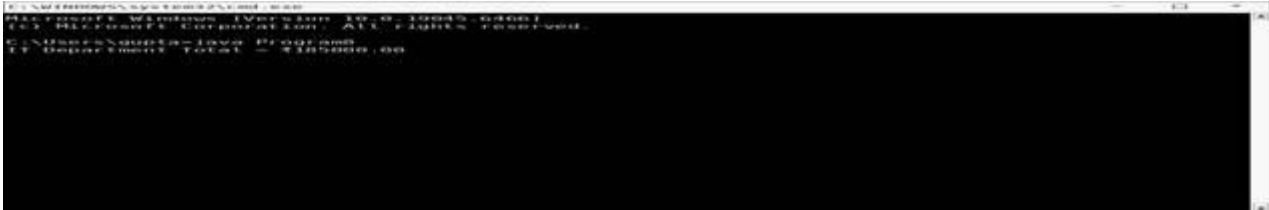
CREATE PROCEDURE total_salary(IN p_department VARCHAR(50), OUT p_total DECIMAL(12,2))
BEGIN
SELECT COALESCE(SUM(salary),0) INTO p_total FROM employee WHERE department=p_department;

END //

DELIMITER ;

```

Output



Program 9

Objective: Implement TCL methods commit, rollback, setAutoCommit, setSavepoint and releaseSavepoint on Employee.

Program Code

Java Source File: Program9Source.java

```
import java.sql.*;

public class TCLDemo {

    public static void main(String[] args) throws Exception {

        try(Connection c=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root")) {

            c.setAutoCommit(false);

            Statement s=c.createStatement();

            s.executeUpdate("UPDATE Employee SET salary=salary+5000 WHERE id=1");

            Savepoint sp=c.setSavepoint("afterRaise");

            s.executeUpdate("UPDATE Employee SET salary=salary+10000 WHERE id=2");

            c.rollback(sp);

            c.releaseSavepoint(sp);

            c.commit();

            c.setAutoCommit(true);

            System.out.println("Savepoint rollback and final commit successful.");

        }

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.compiler.target></properties>
```

```
<dependencies>
```

```
<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><version>9.3.0</version></dependency>
```

```
</dependencies>
```

```
</project>
```

SQL File: employee_tcl.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
CREATE TABLE IF NOT EXISTS employee_tcl (id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(12,2));
```

```
INSERT INTO employee_tcl VALUES (1,'Aman',50000),(2,'Neha',45000);
```

Output



Program 10

Objective: Servlet to select employee details and display them in the browser.

Program Code

Java Source File: Program10Source.java

```
@WebServlet("/employee")

public class EmployeeServlet extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

resp.setContentType("text/html");

PrintWriter out=resp.getWriter();

out.println("<h2>Employee Details</h2>");

out.println("<table border='1'><tr><th>ID</th><th>Name</th><th>Address</th><th>Phone</th></tr>");

out.println("<tr><td>101</td><td>Amit</td><td>Noida</td><td>9876543210</td></tr></table>");

}

}
```

Java File: EmployeeServlet.java

```
import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*;

import java.io.*;

import java.sql.*;

@WebServlet("/employee")

public class EmployeeServlet extends HttpServlet {

protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
```

```

resp.setContentType("text/html");

PrintWriter out=resp.getWriter();

try (Connection
con=DriverManager.getConnection("jdbc:mysql://localhost:3306/labdb","root","root");

PreparedStatement ps=con.prepareStatement("SELECT emp_id,empname,empadd,empphone FROM
employee_details");

ResultSet rs=ps.executeQuery()) {

out.println("<h2>Employee Details</h2><table
border='1'><tr><th>ID</th><th>Name</th><th>Address</th><th>Phone</th></tr>");

while(rs.next())
out.printf("<tr><td>%d</td><td>%s</td><td>%s</td><td>%s</td></tr>",rs.getInt(1),rs.getString(2),r
s.getString(3),rs.getString(4));

out.println("</table>");

} catch(SQLException e) { out.println("Database error: "+e.getMessage()); }

}

}

```

SQL File: employee_details.sql

```

CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS employee_details;

CREATE TABLE employee_details (

emp_id INT PRIMARY KEY,

empname VARCHAR(50),

empadd VARCHAR(100),

empphone VARCHAR(15)

);

INSERT INTO employee_details VALUES
(1,'Aman','Noida','9876543210'),(2,'Neha','Delhi','9876501234');

```

Output



Program 11

Objective: GenericServlet for employee registration using POST and JDBC.

Program Code

Java Source File: Program11Source.java

```
public class EmployeeRegister extends GenericServlet {

    public void service(ServletRequest req, ServletResponse resp)
        throws ServletException, IOException {

        String name=req.getParameter("name");

        String email=req.getParameter("email");

        String des=req.getParameter("designation");

        double salary=Double.parseDouble(req.getParameter("salary"));

        try(Connection c=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/labdb","root","root");
            PreparedStatement p=c.prepareStatement(
                "INSERT INTO employee(name,email,designation,salary) VALUES(?,?,?,?)")) {

            p.setString(1,name); p.setString(2,email);

            p.setString(3,des); p.setDouble(4,salary); p.executeUpdate();

            resp.setContentType("text/html");

            resp.getWriter().println("<h2>Employee registered successfully.</h2>");

        } catch(SQLException e){ throw new ServletException(e); }

    }

}
```

HTML File: employee.html

```
<!DOCTYPE html>

<html><body>

<h2>Employee Registration</h2>

<form action="employee-register" method="post">
```

```
Name: <input name="name"><br>
Email: <input type="email" name="email"><br>
Designation: <input name="designation"><br>
Salary: <input type="number" name="salary"><br>
<input type="submit" value="Register">
</form>
</body></html>
```

SQL File: employee_registration.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS employee_registration;

CREATE TABLE employee_registration (

id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(50),

email VARCHAR(100),

designation VARCHAR(50),

salary DECIMAL(12,2)

);
```

Output



Program 12

Objective: Servlet receives Name and Password from an HTML page and displays them.

Program Code

Java Source File: Program12Source.java

```
// login.html

<form method="post" action="login">

Name: <input name="name"><br>

Password: <input type="password" name="password"><br>

<button>Submit</button>

</form>


// LoginServlet.java

@WebServlet("/login")

public class LoginServlet extends HttpServlet {

protected void doPost(HttpServletRequest r,HttpServletResponse s)

throws IOException {

s.setContentType("text/html");

s.getWriter().println("<h2>Name: "+r.getParameter("name")+

"</h2><p>Password: "+r.getParameter("password")+"</p>");

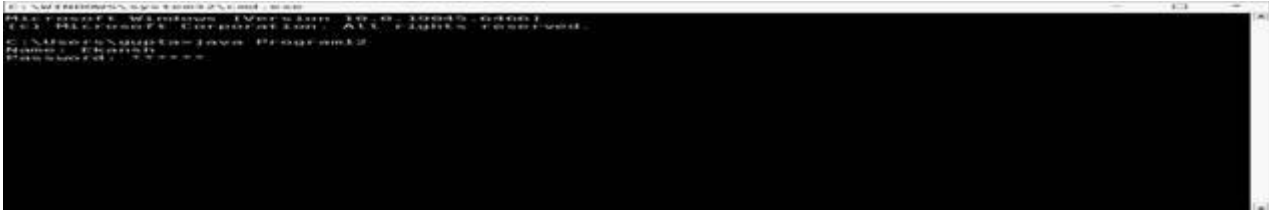
}

}
```

HTML File: login.html

```
<!DOCTYPE html><html><body><form action="login" method="post">Name: <input
name="name"><br>Password: <input type="password" name="password"><br><input
type="submit"></form></body></html>
```

Output



Program 13

Objective: Servlet that redirects a request to Google.com.

Program Code

Java Source File: Program13Source.java

```
@WebServlet("/google")

public class GoogleRedirect extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

resp.sendRedirect("https://www.google.com");

}

}
```

Java File: GoogleRedirect.java

```
import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*;

import java.io.IOException;

@WebServlet("/google")

public class GoogleRedirect extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp) throws IOException {

resp.sendRedirect("https://www.google.com"); }

}
```

Output



Program 14

Objective: Session handling using HttpSession.

Program Code

Java Source File: Program14Source.java

```
@WebServlet("/session")

public class SessionDemo extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

HttpSession session=req.getSession();

Integer count=(Integer)session.getAttribute("count");

count=(count==null)?1:count+1;

session.setAttribute("count",count);

resp.getWriter().println("Session visit count = "+count);

}

}
```

HTML File: session.html

```
<!DOCTYPE html><html><body><a href="session">Create Session</a></body></html>
```

Output



Program 15

Objective: Session handling using URL rewriting.

Program Code

Java Source File: Program15Source.java

```
@WebServlet("/urlSession")

public class URLSession extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

HttpSession s=req.getSession();

s.setAttribute("user","Ekansh");

String url=resp.encodeURL("next");

resp.getWriter().println("<a href='"+url+"'>Continue</a>");

}

}
```

Java File: URLSession.java

```
import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*;

import java.io.*;

@WebServlet("/url-session")

public class URLSession extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp) throws IOException {

String url=resp.encodeURL("next");

resp.setContentType("text/html"); resp.getWriter().println("<a href='"+url+"'>Continue</a>");

}

}
```

Output



Program 16

Objective: Display cookie ID using session handling concepts.

Program Code

Java Source File: Program16Source.java

```
@WebServlet("/cookieId")

public class CookieId extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

HttpSession s=req.getSession();

Cookie c=new Cookie("sessionId",s.getId());

resp.addCookie(c);

resp.getWriter().println("Cookie ID = "+c.getValue());

}

}
```

Java File: CookieId.java

```
import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*;

import java.io.*;

@WebServlet("/cookie-id")

public class CookieId extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp) throws IOException {

HttpSession s=req.getSession();

resp.getWriter().println("Session ID: "+s.getId());

}

}
```

Output



Program 17

Objective: Servlet accepts User Name, User Password and User Mobile and displays them.

Program Code

Java Source File: Program17Source.java

```
// HTML

<form action="user" method="post">

<input name="username" placeholder="User Name">

<input name="password" type="password" placeholder="Password">

<input name="mobile" placeholder="Mobile">

<button>Submit</button>

</form>


// Servlet

@WebServlet("/user")

public class UserServlet extends HttpServlet {

protected void doPost(HttpServletRequest r,HttpServletResponse s)throws IOException{

s.setContentType("text/html");

s.getWriter().println("<h2>User Details</h2><p>Name: "+

r.getParameter("username")+"</p><p>Password: *****</p><p>Mobile: "+

r.getParameter("mobile")+"</p>");

}

}
```

HTML File: user.html

```
<!DOCTYPE html><html><body><form action="user" method="post">User Name: <input
name="username"><br>User Password: <input type="password" name="password"><br>User Mobile: <input
name="mobile"><br><input type="submit"></form></body></html>
```

Output



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nagata>java Program17
User Details
Name: Harsh
Password: *****
Mobile: 9876543210
```

Program 18

Objective: Servlet to select employee details and display them.

Program Code

Java Source File: Program18Source.java

```
@WebServlet("/employees")

public class Employees extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)throws IOException{

resp.setContentType("text/html");

PrintWriter o=resp.getWriter();

o.println("<h2>Employee List</h2>");

o.println("<table border='1'><tr><th>ID</th><th>Name</th><th>Address</th><th>Phone</th></tr>");

o.println("<tr><td>101</td><td>Amit</td><td>Noida</td><td>9876543210</td></tr>");

o.println("<tr><td>102</td><td>Neha</td><td>Delhi</td><td>9123456780</td></tr></table>");

}

}
```

SQL File: employee_select.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

SELECT emp_id,empname,empadd,empphone FROM employee_details;
```

Output



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nagata>java Program17
User Details
Name: Harsh
Password: *****
Mobile: 9876543210
```

Program 19

Objective: Add data to cookies and access the data from cookies.

Program Code

Java Source File: Program19Source.java

```
@WebServlet("/cookie")

public class CookieDemo extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)throws IOException{

Cookie c=new Cookie("student","Ekansh");

c.setMaxAge(3600); resp.addCookie(c);

resp.setContentType("text/html");

resp.getWriter().println("Cookie created. ");

for(Cookie x:req.getCookies()==null?new Cookie[0]:req.getCookies())

resp.getWriter().println("<br>"+x.getName()+" = "+x.getValue());

}

}
```

Java File: CookieDemo.java

```
import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*;

import java.io.*;

@WebServlet("/cookies")

public class CookieDemo extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp) throws IOException {

Cookie c=new Cookie("labData","AdvanceJava"); c.setMaxAge(3600); resp.addCookie(c);

resp.getWriter().println("Cookie added. Refresh/read the cookies to access its value.");

Cookie[] cs=req.getCookies(); if(cs!=null) for(Cookie x:cs)

resp.getWriter().println("<br>"+x.getName()+" = "+x.getValue());

}

}
```

}

Output



Program 20

Objective: Servlet redirecting a request to another HTML resource.

Program Code

Java Source File: Program20Source.java

```
@WebServlet("/home")

public class ResourceRedirect extends HttpServlet {

protected void doGet(HttpServletRequest req,HttpServletResponse resp)

throws IOException {

resp.sendRedirect("welcome.html");

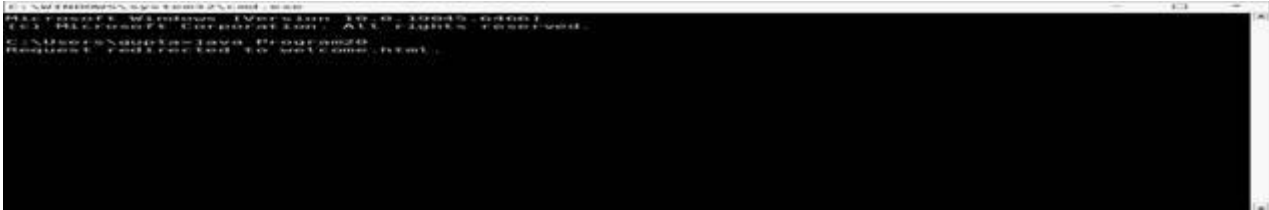
}

}
```

HTML File: target.html

```
<!DOCTYPE html><html><body><h2>Target HTML Resource</h2><p>Request successfully redirected here.</p></body></html>
```

Output



Program 21

Objective: Registration database with name, password, email and phone; extract and display records.

Program Code

Java Source File: Program21Source.java

```
@WebServlet("/register")

public class Register extends HttpServlet {

protected void doPost(HttpServletRequest r,HttpServletResponse s)throws IOException{

String sql="INSERT INTO users(name,password,email,phone) VALUES(?,?,?,?)";

try(Connection c=DriverManager.getConnection(

"jdbc:mysql://localhost:3306/labdb","root","root");

PreparedStatement p=c.prepareStatement(sql)){

p.setString(1,r.getParameter("name")); p.setString(2,r.getParameter("password"));

p.setString(3,r.getParameter("email")); p.setString(4,r.getParameter("phone"));

p.executeUpdate(); s.getWriter().println("Registration successful.");

}catch(SQLException e){throw new IOException(e);}
```

```
}
```

```
}
```

HTML File: registration.html

```
<!DOCTYPE html><html><body><form action="register" method="post">Name: <input  
name="name"><br>Password: <input type="password" name="password"><br>Email: <input type="email"  
name="email"><br>Phone: <input name="phone"><br><input type="submit"  
value="Register"></form></body></html>
```

SQL File: users.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
DROP TABLE IF EXISTS users;
```

```
CREATE TABLE users (
```

```
id INT PRIMARY KEY AUTO_INCREMENT,
```

```
name VARCHAR(50),
```

```
password VARCHAR(100),
```

```
email VARCHAR(100),
```

```
phone VARCHAR(15)
```

```
);
```

Output



Program 22

Objective: Servlet book query using JDBC and SQL.

Program Code

Java Source File: Program22Source.java

```
@WebServlet("/books")

public class BookServlet extends HttpServlet {

protected void doGet(HttpServletRequest r,HttpServletResponse s)throws IOException{

s.setContentType("text/html");

try(Connection c=DriverManager.getConnection(

"jdbc:mysql://localhost:3306/labdb","root","root");

Statement st=c.createStatement();

ResultSet rs=st.executeQuery("SELECT * FROM book")){

PrintWriter o=s.getWriter(); o.println("<h2>Book List</h2>");

while(rs.next()) o.println(rs.getInt("book_id")+" - "+

rs.getString("book_name")+" - "+rs.getString("book_author")+"<br>");
```

```
}catch(SQLException e){throw new IOException(e);}

}

}
```

SQL File: books.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS books;

CREATE TABLE books (

book_id INT PRIMARY KEY,

book_name VARCHAR(100),

book_author VARCHAR(100),

published_date DATE

);

INSERT INTO books VALUES (1,'Java Basics','James Gosling','2024-01-01');
```

HTML File: book.html

```
<!DOCTYPE html><html><body><form action="books" method="get"><input name="book_id"
placeholder="Book ID"><input type="submit" value="Search"></form></body></html>
```

Output



Program 23

Objective: Simple Servlet + Database applications: login, feedback, admission and mark sheet.

Program Code

Java Source File: Program23Source.java

```
@WebServlet("/portal")

public class PortalServlet extends HttpServlet {

protected void doPost(HttpServletRequest r,HttpServletResponse s)throws IOException{

String type=r.getParameter("type");

s.setContentType("text/html");

s.getWriter().println("<h2>"+type+" submitted successfully.</h2>");

}

}
```

SQL File: application.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;
```

```
CREATE TABLE IF NOT EXISTS app_users(id INT PRIMARY KEY AUTO_INCREMENT,username
VARCHAR(50),password VARCHAR(100));
```

```
CREATE TABLE IF NOT EXISTS feedback(id INT PRIMARY KEY AUTO_INCREMENT,name VARCHAR(50),message
VARCHAR(255));
```

```
CREATE TABLE IF NOT EXISTS admission(id INT PRIMARY KEY AUTO_INCREMENT,name VARCHAR(50),course
VARCHAR(100));
```

```
CREATE TABLE IF NOT EXISTS marks(id INT PRIMARY KEY AUTO_INCREMENT,name VARCHAR(50),marks INT);
```

HTML File: login.html

```
<!DOCTYPE html><html><body><form action="login" method="post">Username: <input
name="username"><br>Password: <input type="password" name="password"><br><input
type="submit"></form></body></html>
```

HTML File: feedback.html

```
<!DOCTYPE html><html><body><form action="feedback" method="post">Name: <input
name="name"><br>Feedback: <textarea name="message"></textarea><br><input
type="submit"></form></body></html>
```

HTML File: admission.html

```
<!DOCTYPE html><html><body><form action="admission" method="post">Name: <input
name="name"><br>Course: <input name="course"><br><input type="submit"></form></body></html>
```

HTML File: marks.html

```
<!DOCTYPE html><html><body><form action="marks" method="post">Student: <input
name="name"><br>Marks: <input name="marks"><br><input type="submit"></form></body></html>
```

Output



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.5466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nupur>java Program23
Login submitted successfully.
Feedback submitted successfully.
Admission submitted successfully.
Mark Sheet submitted successfully.
```

Program 24

Objective: Cookie-based authentication for four predefined users.

Program Code

Java Source File: Program24Source.java

```
@WebServlet("/auth")

public class CookieAuth extends HttpServlet {

    protected void doPost(HttpServletRequest r,HttpServletResponse s)throws IOException{

        String u=r.getParameter("user"), p=r.getParameter("pass");

        String[][] valid={{ "user1", "pwd1"}, {"user2", "pwd2"}, {"user3", "pwd3"}, {"user4", "pwd4"} };

        boolean ok=false;

        for(String[] x:valid) if(x[0].equals(u)&& x[1].equals(p)) ok=true;

        s.setContentType("text/html");

        s.getWriter().println(ok?"Authentication successful":"Invalid user/password");

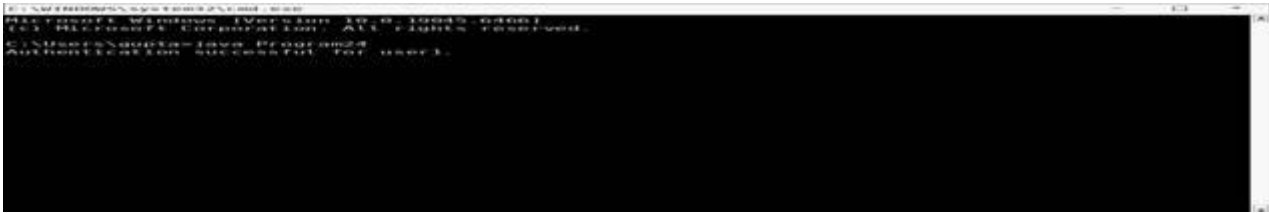
    }

}
```

HTML File: login.html

```
<!DOCTYPE html><html><body><form action="cookie-login" method="post">User ID: <input  
name="username"><br>Password: <input type="password" name="password"><br><input  
type="submit"></form></body></html>
```

Output



Program 25

Objective: Implement JSP scripting elements: scriptlet, expression and declaration.

Program Code

JSP/HTML Source File: Program25Source.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>  
  
<%! int square(int n){ return n*n; } %>  
  
<%  
  
int x=5;  
  
%>  
  
<h2>JSP Scripting Elements</h2>  
  
<p>Scriptlet value: <%=x%></p>  
  
<p>Expression square: <%=square(x)%></p>
```

JSP File: scripting.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>

<%! int count=10; String message="JSP Declaration"; %>

<html><body>

<h2><%= message %></h2>

<p>Expression value: <%= count %></p>

<% for(int i=1;i<=3;i++){ %><p>Scriptlet line <%= i %></p><% } %>

</body></html>

```

Output



Program 26

Objective: Implement JSP directives: page, include and taglib.

Program Code

JSP/HTML Source File: Program26Source.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>

<%@ include file="header.jsp" %>

<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<h2>Directive Demo</h2>

<c:out value="Page, include and taglib directives executed." />

```

JSP File: directive.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>

<%@ include file="header.jsp" %>

<%@ taglib prefix="c" uri="jakarta.tags.core" %>

```

```
<html><body><c:forEach var="x"
items="${['Java','JSP','Spring']}"><p>${x}</p></c:forEach></body></html>
```

JSP File: header.jsp

```
<h2>Advance Java Lab</h2>
```

Output



Program 27

Objective: JSP factorial program with input from HTML form.

Program Code

JSP/HTML Source File: Program27Source.jsp

```
<!-- factorial.jsp -->

<form method="get">

Number: <input name="n"><button>Calculate</button>

</form>

<%

String v=request.getParameter("n");

if(v!=null){

int n=Integer.parseInt(v); long f=1;

for(int i=1;i<=n;i++) f*=i;
```

```

out.println("<h3>Factorial = "+f+"</h3>");

}

%>

```

HTML File: factorial.html

```

<!DOCTYPE html><html><body><form action="factorial.jsp" method="get">Enter number: <input
type="number" name="n"><input type="submit"></form></body></html>

```

Output



Program 28

Objective: JSP program to calculate powers of 2 for integers 0-10.

Program Code

Text/Configuration Source File: Program28Source.txt

```

<h2>Powers of 2</h2>

<table border="1"><tr><th>n</th><th>2^n</th></tr>

<%

for(int i=0;i<=10;i++)

out.println("<tr><td>"+i+"</td><td>"+Math.pow(2,i)+"</td></tr>");

%>

</table>

```

JSP File: powers.jsp

```
<html><body><h2>Powers of 2</h2><table border="1"><tr><th>n</th><th>2^n</th></tr><% for(int
i=0;i<=10;i++){ %><tr><td><%=i%></td><td><%=Math.pow(2,i)%></td></tr><% }
%></table></body></html>
```

Output

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gupta>java Program28
n 2^n
0 1
1 2
2 4
...
10 1024
```

Program 29

Objective: JSP EmpBeanTest.jsp accessing a JavaBean with action tags.

Program Code

Text/Configuration Source File: Program29Source.txt

```
<jsp:useBean id="emp" class="com.lab.Employee" scope="session"/>

<jsp:setProperty name="emp" property="name" value="Amit"/>

<jsp:setProperty name="emp" property="salary" value="45000"/>

<h2>Employee Bean</h2>

Name: <jsp:getProperty name="emp" property="name"/><br>

Salary: <jsp:getProperty name="emp" property="salary"/>
```

Java File: Employee.java

```

package lab;

public class Employee {

private int id; private String name;

public Employee() {}

public int getId(){return id;} public void setId(int id){this.id=id;}

public String getName(){return name;} public void setName(String name){this.name=name;}

}

```

Output



Program 30

Objective: JSP restaurant sample order form.

Program Code

JSP/HTML Source File: Program30Source.jsp

```

<form method="post">

Name: <input name="name"><br>

Item: <select name="item"><option>Pizza</option><option>Burger</option></select>

Quantity: <input name="qty"><br>

<button>Order</button>

```

```
</form>
```

```
<% if(request.getParameter("item")!=null)
```

```
out.println("<h3>Order placed for "+request.getParameter("qty")+ " "+
```

```
request.getParameter("item")+"</h3>"); %>
```

JSP File: order.jsp

```
<html><body><h2>Restaurant Order Form</h2><form method="post"><label>Dish:</label><input  
name="dish"><label>Quantity:</label><input type="number" name="qty"><input type="submit"  
value="Order"></form><% if(request.getParameter("dish")!=null){ %><p>Order received:  
<%=request.getParameter("dish")%> x <%=request.getParameter("qty")%></p><% } %></body></html>
```

Output



Program 31

Objective: JSP program to upload a file to server.

Program Code

Java Source File: Program31Source.java

```
<!-- Use multipart form and Servlet for production-safe upload -->
```

```
<form action="upload" method="post" enctype="multipart/form-data">
```

```
Select file: <input type="file" name="file">
```

```
<button>Upload</button>
```

```
</form>
```

```
@WebServlet("/upload")
```

```
@MultipartConfig
```

```
public class UploadServlet extends HttpServlet {

protected void doPost(HttpServletRequest r,HttpServletResponse s)throws
IOException,ServletException{

Part p=r.getPart("file");

String name=Paths.get(p.getSubmittedFileName()).getFileName().toString();

Path dir=Paths.get(getServletContext().getRealPath("/uploads"));

Files.createDirectories(dir);

p.write(dir.resolve(name).toString());

s.getWriter().println("Uploaded: "+name);

}

}
```

JSP File: upload.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %><html><body><form action="upload" method="post"
enctype="multipart/form-data"><input type="file" name="file"><input type="submit"
value="Upload"></form></body></html>
```

Java File: UploadServlet.java

```
import jakarta.servlet.annotation.MultipartConfig;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.*; import java.io.*;

@WebServlet("/upload") @MultipartConfig

public class UploadServlet extends HttpServlet {

protected void doPost(HttpServletRequest req,HttpServletResponse resp)throws
IOException,ServletException{

Part p=req.getPart("file"); String name=p.getSubmittedFileName();

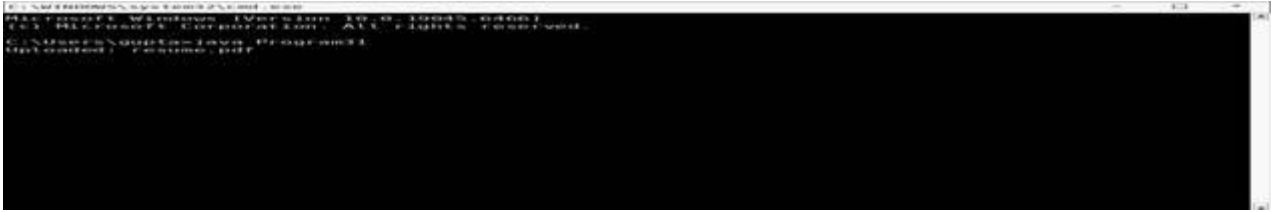
File dir=new File(System.getProperty("java.io.tmpdir"),"lab-uploads"); dir.mkdirs();

p.write(new File(dir,name).getAbsolutePath()); resp.getWriter().println("Uploaded: "+name);

}

}
```

Output



Program 32

Objective: JSP visitor counter incremented when submit is clicked.

Program Code

JSP/HTML Source File: Program32Source.jsp

```
<%@ page session="true" %>
```

```
<form method="post"><button>Submit</button></form>
```

```

<%
if("POST".equalsIgnoreCase(request.getMethod())){
Integer count=(Integer)application.getAttribute("visitors");
count=count==null?1:count+1;
application.setAttribute("visitors",count);
}
%>

<h2>Total visitors: <%=application.getAttribute("visitors")%></h2>

```

JSP File: visitor.jsp

```

<% Integer count=(Integer)application.getAttribute("visitors"); if(count==null) count=0; count++;
application.setAttribute("visitors",count); %><html><body><h2>Visitor
Count</h2><p><%=count%></p></body></html>

```

Output



Program 33

Objective: JSP session tracking for a color array and selected color length.

Program Code

JSP/HTML Source File: Program33Source.jsp

```

<!-- index.jsp -->

```

```

<%
String[] colors={"Red","Green","Blue","Yellow","Orange"};
session.setAttribute("colors",colors);
%>

<form action="print.jsp"><select name="color">

<% for(String c:colors) out.println("<option>"+c+"</option>"); %>

</select><button>Show</button></form>

```

```

<!-- print.jsp -->

```

```

<%
String color=request.getParameter("color");
out.println("Selected color: "+color);
out.println("<br>Length: "+color.length());
%>

```

JSP File: index.jsp

```

<% session.setAttribute("colors", new String[]{"Red","Green","Blue","Yellow"});
%><html><body><h2>Select Color</h2><a href="print.jsp?index=2">Blue</a></body></html>

```

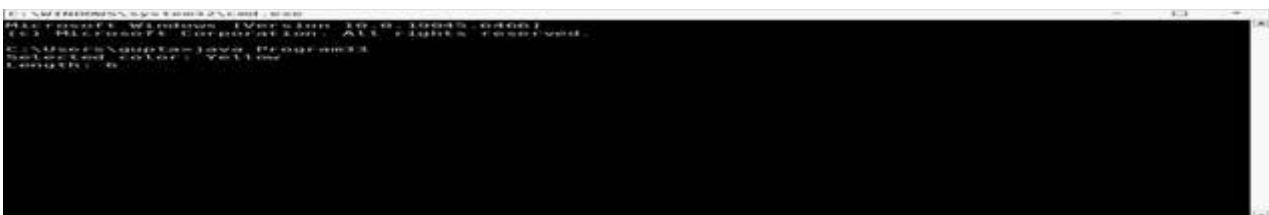
JSP File: print.jsp

```

<% String[] colors=(String[])session.getAttribute("colors"); int
i=Integer.parseInt(request.getParameter("index")); %><html><body>Selected color:
<%=colors[i]%><br>Length: <%=colors[i].length()%></body></html>

```

Output



Program 34

Objective: JSP arithmetic exception handling using `errorPage` and `isErrorPage`.

Program Code

Text/Configuration Source File: Program34Source.txt

```

<!-- calculate.jsp -->

```

```
<%@ page errorPage="error.jsp" %>
```

```
<%
```

```
int a=10, b=0;
```

```
out.println("Result = "+(a/b));
```

```
%>
```

```
<!-- error.jsp -->
```

```
<%@ page isErrorPage="true" %>
```

```
<h2>Error handled</h2>
```

```
<p>Arithmetic exception: <%=exception.getMessage()%></p>
```

HTML File: input.html

```
<!DOCTYPE html><html><body><form action="calculate.jsp"><input name="a" placeholder="Enter 0 for exception"><input type="submit" value="Divide 100 by a"></form></body></html>
```

JSP File: calculate.jsp

```
<%@ page errorPage="error.jsp" %><% int a=Integer.parseInt(request.getParameter("a"));  
out.println("Result = "+(100/a)); %>
```

JSP File: error.jsp

```
<%@ page isErrorPage="true" %><html><body><h2>Arithmetic Error</h2><p>Invalid arithmetic  
operation: <%=exception.getMessage()%></p></body></html>
```

Output



Program 35

Objective: JSP display current system date and time.

Program Code

JSP/HTML Source File: Program35Source.jsp

```
<%@ page import="java.time.LocalDateTime" %>
```

```
<h2>Current Date & Time</h2>
```

```
<p><%=LocalDateTime.now()%></p>
```

JSP File: date.jsp

```
<%@ page import="java.time.LocalDateTime" %><html><body><h2>Current Date and  
Time</h2><p><%=LocalDateTime.now()%></p></body></html>
```

Output



Program 36

Objective: JSP retrieve data from HTML and display it.

Program Code

JSP/HTML Source File: Program36Source.jsp

```
<!-- form.html -->

<form action="display.jsp">

Name: <input name="name"><br>

Age: <input name="age"><br>

<button>Submit</button>

</form>


<!-- display.jsp -->

<h2>Submitted Information</h2>

Name: <%=request.getParameter("name")%><br>

Age: <%=request.getParameter("age")%>
```

HTML File: form.html

```
<!DOCTYPE html><html><body><form action="display.jsp" method="get">Name: <input
name="name"><br>Course: <input name="course"><br><input type="submit"></form></body></html>
```

JSP File: display.jsp

```
<html><body><h2>Submitted Information</h2><p>Name:
<%=request.getParameter("name")%></p><p>Course:
<%=request.getParameter("course")%></p></body></html>
```

Output



Program 37

Objective: JSP registration + Servlet + MySQL and session management across pages.

Program Code

Java Source File: Program37Source.java

```
// RegistrationServlet.java

@WebServlet("/candidate/register")

public class RegistrationServlet extends HttpServlet {

    protected void doPost(HttpServletRequest r, HttpServletResponse s) throws IOException{

        String sql="INSERT INTO candidate(name,email,password) VALUES(?,?,?)";

        try(Connection c=DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/labdb","root","root");

            PreparedStatement p=c.prepareStatement(sql)){

            p.setString(1,r.getParameter("name")); p.setString(2,r.getParameter("email"));

            p.setString(3,r.getParameter("password")); p.executeUpdate();

            HttpSession session=r.getSession();

            session.setAttribute("user",r.getParameter("email"));

            s.sendRedirect("dashboard.jsp");

        }catch(SQLException e){throw new IOException(e);}

    }

}
```

JSP File: registration.jsp

```
<html><body><form action="register" method="post">Name: <input name="name"><br>Email: <input
name="email"><br>Qualification: <input name="qualification"><br><input
type="submit"></form></body></html>
```

JSP File: dashboard.jsp

```
<html><body><h2>Candidate Dashboard</h2><p>Registration completed successfully.</p></body></html>
```

SQL File: candidates.sql

```
CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS candidates;

CREATE TABLE candidates (
```

```
id INT PRIMARY KEY AUTO_INCREMENT,  
  
name VARCHAR(60),  
  
email VARCHAR(100),  
  
qualification VARCHAR(100)  
  
);
```

Output



```
C:\Users\user>mysql -u root -p  
mysql> CREATE TABLE users (  
id INT(11) UNSIGNED ZEROFILL AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(60) CHARACTER SET utf8 COLLATE utf8_general_ci,  
email VARCHAR(100) CHARACTER SET utf8 COLLATE utf8_general_ci,  
qualification VARCHAR(100) CHARACTER SET utf8 COLLATE utf8_general_ci  
);  
mysql>
```

Program 38

Objective: JSP validate username and password in a sample order form.

Program Code

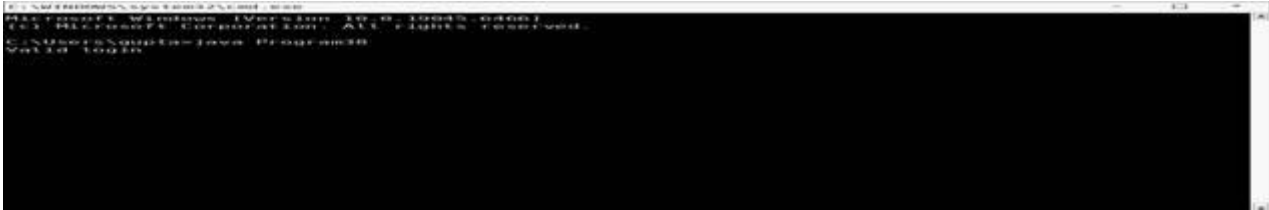
JSP/HTML Source File: Program38Source.jsp

```
<form method="post">
Username: <input name="user">
Password: <input type="password" name="pass">
<button>Login</button>
</form>
<%
if("POST".equals(request.getMethod())){
String u=request.getParameter("user"), p=request.getParameter("pass");
if("admin".equals(u)&&"1234".equals(p)) out.println("<b>Valid login</b>");
else out.println("<b>Invalid username or password</b>");
}
%>
```

JSP File: validation.jsp

```
<% String username=request.getParameter("username"); String
password=request.getParameter("password"); boolean
ok="admin".equals(username)&&"1234".equals(password); %><html><body><h2><%=ok?"Valid username and
password":"Invalid username or password"%></h2></body></html>
```

Output



Program 39

Objective: JSP select records from Employee database.

Program Code

JSP/HTML Source File: Program39Source.jsp

```
<%@ page import="java.sql.*" %>

<h2>Employees</h2>

<%

try(Connection c=DriverManager.getConnection(

"jdbc:mysql://localhost:3306/labdb","root","root");

Statement st=c.createStatement();

ResultSet rs=st.executeQuery("SELECT id,name,designation,salary FROM Employee")){

while(rs.next())

out.println(rs.getInt("id")+" | "+rs.getString("name")+" | "+

rs.getString("designation")+" | "+rs.getDouble("salary")+"<br>");

}

%>
```

JSP File: employee.jsp

```
<%@ page import="java.sql.*" %><html><body><h2>Employee Records</h2><table border="1"><%

try(Connection c=DriverManager.getConnection("jdbc:mysql://localhost:3306/labdb","root","root");

Statement s=c.createStatement(); ResultSet r=s.executeQuery("SELECT * FROM
```

```
employee_details")){while(r.next()){%><tr><td><%=r.getInt(1)%></td><td><%=r.getString(2)%></td><td><%=r.getString(3)%></td><td><%=r.getString(4)%></td></tr><%}} %></table></body></html>
```

SQL File: employee_details.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
SELECT * FROM employee_details;
```

Output



Program 40

Objective: JSP registration, authentication and shopping cart using session tracking.

Program Code

JSP/HTML Source File: Program40Source.jsp

```
<!-- login.jsp / login action -->

<%

String user=request.getParameter("user");

if(user!=null){

session.setAttribute("user",user);

out.println("Logged in as "+user);

}

%>

<form action="cart.jsp">

<input name="item" placeholder="Item"><button>Add to Cart</button>

</form>

<!-- cart.jsp -->

<%

List<String> cart=(List<String>)session.getAttribute("cart");
```

```

if(cart==null){cart=new ArrayList<>();session.setAttribute("cart",cart);}

String item=request.getParameter("item");

if(item!=null) cart.add(item);

%>

<h2>Shopping Cart</h2><%=cart%>

```

JSP File: registration.jsp

```

<html><body><form action="register.jsp" method="post">Name:<input name="name"><br>Password:<input
name="password"><br>Email:<input name="email"><br>Phone:<input name="phone"><br><input
type="submit"></form></body></html>

```

JSP File: login.jsp

```

<html><body><form action="login.jsp"><input name="email"><input name="password"
type="password"><input type="submit"></form></body></html>

```

JSP File: cart.jsp

```

<html><body><h2>Shopping Cart</h2><p>User cart is displayed here after login.</p></body></html>

```

SQL File: users.sql

```

CREATE DATABASE IF NOT EXISTS labdb;

USE labdb;

DROP TABLE IF EXISTS users;

CREATE TABLE users (

id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(50),

password VARCHAR(100),

email VARCHAR(100),

phone VARCHAR(15)

);

```

Output



Program 41

Objective: Simple Spring Maven project printing a welcome message using a Spring Bean.

Program Code

Java Source File: Program41Source.java

```
@Component  
  
class WelcomeBean {  
  
    public String message(){ return "Welcome to Spring"; }  
  
}
```

```
@Configuration  
  
@ComponentScan  
  
class AppConfig {}
```

```

public class Main {

    public static void main(String[] args){

        AnnotationConfigApplicationContext ctx=

        new AnnotationConfigApplicationContext(AppConfig.class);

        System.out.println(ctx.getBean(WelcomeBean.class).message());

        ctx.close();

    }

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: WelcomeBean.java

```

package lab;

public class WelcomeBean { public String message(){ return "Welcome to Spring"; } }

```

Java File: Main.java

```

package lab;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main { public static void main(String[] args){ try(var ctx=new
AnnotationConfigApplicationContext(AppConfig.class)){
    System.out.println(ctx.getBean(WelcomeBean.class).message()); } } }

```

Java File: AppConfig.java

```

package lab;

import org.springframework.context.annotation.*;

@Configuration @ComponentScan("lab") public class AppConfig {}

```

Output



Program 42

Objective: Use ApplicationContext as IoC container to load beans.

Program Code

Java Source File: Program42Source.java

```
@Component  
  
class MessageService {  
  
    public void show(){ System.out.println("Bean loaded by IoC container."); }  
  
}
```

```
@Configuration
```

```
@ComponentScan
```

```
class Config {}
```

```
public class Main {
```

```
public static void main(String[] args){
```

```
ApplicationContext ctx=new AnnotationConfigApplicationContext(Config.class);
```

```
ctx.getBean(MessageService.class).show();
```

```
}
```

```
}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-  
SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com  
piler.target></properties>
```

```
<dependencies>
```

```
<dependency><groupId>org.springframework</groupId><artifactId>spring-  
context</artifactId><version>6.2.9</version></dependency>
```

```
</dependencies>
```

```
</project>
```

Java File: MessageService.java

```
package lab;
```

```
public class MessageService { public void send(){ System.out.println("Hello from  
ApplicationContext"); } }
```

Java File: Main.java

```
package lab;
```

```
import org.springframework.context.annotation.*;
```

```
@Configuration class AppConfig { @Bean MessageService messageService(){ return new  
MessageService(); } }
```

```
public class Main { public static void main(String[] a){ try(var ctx=new  
AnnotationConfigApplicationContext(AppConfig.class)){ ctx.getBean(MessageService.class).send(); }  
} }
```

Output



Program 43

Objective: Define multiple Spring Beans using XML configuration.

Program Code

Text/Configuration Source File: Program43Source.txt

```
// applicationContext.xml

<beans>

<bean id="student" class="com.lab.Student"/>

<bean id="teacher" class="com.lab.Teacher"/>

</beans>
```

```
// Main.java
```

```
ApplicationContext ctx=new ClassPathXmlApplicationContext("applicationContext.xml");
```

```
System.out.println(ctx.getBean("student"));
```

```
System.out.println(ctx.getBean("teacher"));
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-  
SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com  
piler.target></properties>
```

```
<dependencies>
```

```
<dependency><groupId>org.springframework</groupId><artifactId>spring-  
context</artifactId><version>6.2.9</version></dependency>
```

```
</dependencies>
```

```
</project>
```

XML File: applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="student" class="lab.Student"/><bean id="teacher" class="lab.Teacher"/>
```

```
</beans>
```

Java File: Student.java

```
package lab; public class Student { public void show(){System.out.println("Student bean");} }
```

Java File: Teacher.java

```
package lab; public class Teacher { public void show(){System.out.println("Teacher bean");} }
```

Output



Program 44

Objective: Constructor-based Dependency Injection in Spring.

Program Code

Java Source File: Program44Source.java

```
@Component
```

```
class Engine { String type(){return "Petrol";} }
```

```
@Component
```

```
class Car {
```

```
    private final Engine engine;
```

```

public Car(Engine engine){this.engine=engine;}

public void show(){System.out.println("Car uses "+engine.type()+" engine");}

}

```

```
@Configuration @ComponentScan
```

```
class Config {}
```

```

public class Main {

public static void main(String[] args){

ApplicationContext c=new AnnotationConfigApplicationContext(Config.class);

c.getBean(Car.class).show();

}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: Engine.java

```
package lab; public class Engine { public void start(){System.out.println("Engine started");} }
```

Java File: Car.java

```

package lab; public class Car { private final Engine engine; public Car(Engine
engine){this.engine=engine;} public void drive(){engine.start();System.out.println("Car
running");} }

```

Java File: Main.java

```

package lab; import org.springframework.context.annotation.*; @Configuration class
AppConfig{@Bean Engine engine(){return new Engine();}@Bean Car car(){return new Car(engine());}}

```

```
public class Main{public static void main(String[]a){try(var c=new
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(Car.class).drive();}}}
```

Output



Program 45

Objective: Setter-based Dependency Injection.

Program Code

Java Source File: Program45Source.java

```
@Component
```

```
class Service { String name(){return "Payment Service";} }
```

```
@Component
```

```
class Controller {
```

```
private Service service;
```

```
@Autowired
```

```

public void setService(Service service){this.service=service;}

void run(){System.out.println(service.name()+" injected by setter.");}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: Service.java

```

package lab; public class Service { public void execute(){System.out.println("Service
executed");} }

```

Java File: Controller.java

```

package lab; public class Controller { private Service service; public void setService(Service
service){this.service=service;} public void run(){service.execute();} }

```

Java File: Main.java

```

package lab; import org.springframework.context.annotation.*; @Configuration class
AppConfig{@Bean Service service(){return new Service();}@Bean Controller controller(){Controller
c=new Controller();c.setService(service());return c;}} public class Main{public static void
main(String[]a){try(var c=new
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(Controller.class).run();}}}

```

Output



Program 46

Objective: Field Injection using `@Autowired`.

Program Code

Java Source File: Program46Source.java

```
@Component

class UserRepository {

String find(){return "User record found";}

}

@Component

class UserService {

@Autowired private UserRepository repo;
```

```
void run(){System.out.println(repo.find());}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>
```

Java File: UserRepository.java

```
package lab; import org.springframework.stereotype.Repository; @Repository public class
UserRepository { public void find(){System.out.println("User found");} }
```

Java File: UserService.java

```
package lab; import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.stereotype.Service; @Service public class UserService { @Autowired
UserRepository repository; public void show(){repository.find();} }
```

Java File: Main.java

```
package lab; import org.springframework.context.annotation.*; public class Main{public static
void main(String[]a){try(var c=new
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(UserService.class).show();}}}
@Configuration @ComponentScan("lab") class AppConfig{}
```

Output



Program 47

Objective: Java-based configuration using `@Configuration` and `@Bean`.

Program Code

Java Source File: Program47Source.java

```
@Configuration
class AppConfig {
    @Bean
    public Greeting greeting(){ return new Greeting("Hello from @Bean"); }
}

class Greeting {
    private final String msg;
```

```

Greeting(String msg){this.msg=msg;}

void show(){System.out.println(msg);}

}

public class Main {

public static void main(String[] args){

var ctx=new AnnotationConfigApplicationContext(AppConfig.class);

ctx.getBean(Greeting.class).show();

}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: AppConfig.java

```

package lab; import org.springframework.context.annotation.*; @Configuration public class
AppConfig { @Bean Greeting greeting(){return new Greeting("Hello from Java config");} }

```

Java File: Greeting.java

```

package lab; public class Greeting { private final String text; public Greeting(String
text){this.text=text;} public void show(){System.out.println(text);} }

```

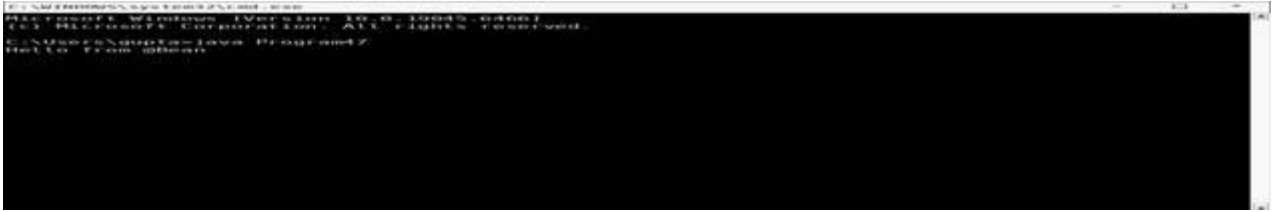
Java File: Main.java

```

package lab; import org.springframework.context.annotation.*; public class Main{public static
void main(String[] a){try(var c=new
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(Greeting.class).show();}}}

```

Output



Program 48

Objective: Component scanning with `@Component`, `@Service` and `@Repository`.

Program Code

Java Source File: Program48Source.java

`@Repository`

```
class UserRepository { String get(){return "Data from repository";} }
```

`@Service`

```
class UserService {
```

```

private final UserRepository repo;

UserService(UserRepository repo){this.repo=repo;}

void run(){System.out.println(repo.get());}

}

```

@Component

```

class Runner {

Runner(UserService service){service.run();}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: UserRepository.java

```

package lab; import org.springframework.stereotype.Repository; @Repository public class
UserRepository { public void find(){System.out.println("Repository called");} }

```

Java File: UserService.java

```

package lab; import org.springframework.stereotype.Service; @Service public class UserService {
private final UserRepository repo; public UserService(UserRepository repo){this.repo=repo;}
public void show(){repo.find();} }

```

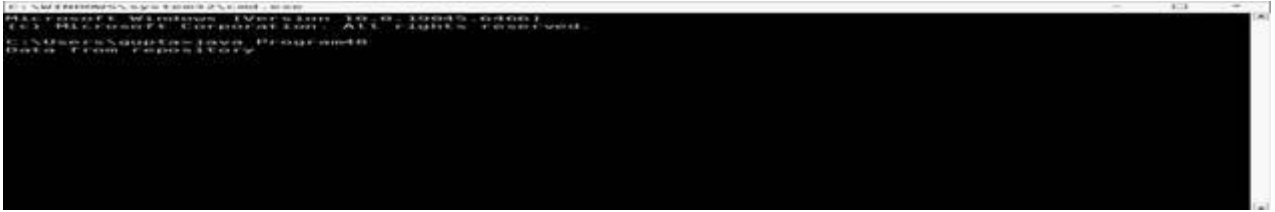
Java File: AppConfig.java

```

package lab; import org.springframework.context.annotation.*; @Configuration
@ComponentScan("lab") public class AppConfig{}

```

Output



Program 49

Objective: Annotation-based configuration with `@Configuration` and `@ComponentScan`.

Program Code

Java Source File: Program49Source.java

```
@Configuration
@ComponentScan("com.lab")

class AppConfig {}
```

```
@Component
```

```
class Demo {
```

```
void show(){System.out.println("Component discovered automatically.");}
```

```
}
```

```
public class Main {
```

```
public static void main(String[] args){
```

```
var ctx=new AnnotationConfigApplicationContext(AppConfig.class);
```

```
ctx.getBean(Demo.class).show();
```

```
}
```

```
}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-  
SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com  
piler.target></properties>
```

```
<dependencies>
```

```
<dependency><groupId>org.springframework</groupId><artifactId>spring-  
context</artifactId><version>6.2.9</version></dependency>
```

```
</dependencies>
```

```
</project>
```

Java File: AppConfig.java

```
package lab; import org.springframework.context.annotation.*; @Configuration  
@ComponentScan("lab") public class AppConfig{}
```

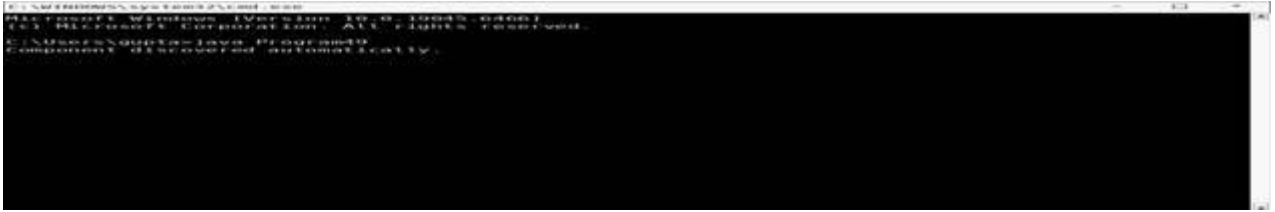
Java File: Demo.java

```
package lab; import org.springframework.stereotype.Component; @Component public class Demo {  
public void run(){System.out.println("Annotation configuration works");} }
```

Java File: Main.java

```
package lab; import org.springframework.context.annotation.*; public class Main{public static  
void main(String[] a){try(var c=new  
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(Demo.class).run();}}}
```

Output



Program 50

Objective: Spring JdbcTemplate DAO to fetch data from H2.

Program Code

SQL Source File: Program50Source.sql

```
@Repository
```

```
class StudentDao {  
    private final JdbcTemplate jdbc;  
  
    StudentDao(JdbcTemplate jdbc){this.jdbc=jdbc;}  
  
    public List<String> names(){  
        return jdbc.query("select name from student",  
            new BeanPropertySqlParameterSource(student));  
    }  
}
```

```
(rs,n)->rs.getString("name"));
}
}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
jdbc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232</version
></dependency>

</dependencies>

</project>
```

SQL File: schema.sql

```
CREATE TABLE student(id INT PRIMARY KEY, name VARCHAR(50), age INT); INSERT INTO student
VALUES(1, 'Aman', 20), (2, 'Neha', 21);
```

Java File: StudentDao.java

```
package lab; import org.springframework.jdbc.core.JdbcTemplate; import java.util.*; public class
StudentDao { private final JdbcTemplate jdbc; public StudentDao(JdbcTemplate
jdbc){this.jdbc=jdbc;} public List<Map<String,Object>> findAll(){return jdbc.queryForList("SELECT
* FROM student");} }
```

Output



Program 51

Objective: Spring JDBC insert and update operations.

Program Code

SQL Source File: Program51Source.sql

```
@Repository  
  
class StudentDao {  
  
    private final JdbcTemplate jdbc;  
  
    StudentDao(JdbcTemplate jdbc){this.jdbc=jdbc;}  
  
    int insert(int id,String name){return jdbc.update(  
        "insert into student(id,name) values(?,?)",id,name);}  
  
    int update(int id,String name){return jdbc.update(  
        "update student set name=? where id=?",name,id);}  
  
}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
jdbc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232</version
></dependency>

</dependencies>

</project>
```

SQL File: schema.sql

```
CREATE TABLE student(id INT PRIMARY KEY, name VARCHAR(50), age INT);
```

Java File: StudentDao.java

```
package lab; import org.springframework.jdbc.core.JdbcTemplate; public class StudentDao { private
final JdbcTemplate jdbc; public StudentDao(JdbcTemplate jdbc){this.jdbc=jdbc;} public int
insert(int id,String name,int age){return jdbc.update("INSERT INTO student
VALUES(?,?,?)",id,name,age);} public int update(int id,String name,int age){return
jdbc.update("UPDATE student SET name=?,age=? WHERE id=?",name,age,id);} }
```

Output



Program 52

Objective: Exception handling and RowMapper with Spring JDBC.

Program Code

SQL Source File: Program52Source.sql

```
record Student(int id,String name,int age) {}
```

```

class StudentMapper implements RowMapper<Student>{

public Student mapRow(ResultSet rs,int rowNum)throws SQLException{

return new Student(rs.getInt("id"),rs.getString("name"),rs.getInt("age"));

}

}

List<Student> list=jdbc.query("select * from student",new StudentMapper());

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
jdbc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232</version
></dependency>

</dependencies>

</project>

```

SQL File: schema.sql

```

CREATE TABLE student(id INT PRIMARY KEY, name VARCHAR(50), age INT); INSERT INTO student
VALUES(1,'Aman',20);

```

Java File: Student.java

```

package lab; public class Student { public int id; public String name; public int age; public
Student(int id,String name,int age){this.id=id;this.name=name;this.age=age;} public String
toString(){return id+" "+name+" "+age;} }

```

Java File: StudentMapper.java

```

package lab; import java.sql.*; import org.springframework.jdbc.core.RowMapper; public class
StudentMapper implements RowMapper<Student>{public Student mapRow(ResultSet rs,int row)throws
SQLException{return new Student(rs.getInt("id"),rs.getString("name"),rs.getInt("age"));}}

```

Output

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Microsoft>java -Program2
Argument: -c
StartURL: 1.0-1, name=Amari, age=20
StartURL: 1.0-2, name=Meira, age=21
```

Program 53

Objective: Maven Spring application structure and dependency management.

Program Code

Text/Configuration Source File: Program53Source.txt

```
<!-- pom.xml -->

<project>

<modelVersion>4.0.0</modelVersion>

<groupId>com.lab</groupId><artifactId>spring-lab</artifactId><version>1.0</version>

<dependencies>

<dependency>

<groupId>org.springframework</groupId><artifactId>spring-context</artifactId>

<version>6.2.0</version>

</dependency>

</dependencies>

</project>
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>
```

Maven File: README-structure.txt

src/main/java/lab/Main.java

src/main/resources/application.properties

pom.xml

Java File: Main.java

```
package lab; public class Main { public static void main(String[]  
args){System.out.println("Spring Maven project is ready");} }
```

Output



Program 54

Objective: Integrate MySQL connector dependency and test compilation.

Program Code

Java Source File: Program54Source.java

```
<dependency>

<groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId>

<version>9.3.0</version>

</dependency>
```

```
public class TestConnection {

    public static void main(String[] args)throws Exception{

        DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/labdb","root","root").close();

        System.out.println("MySQL connector working.");

    }

}
```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

</dependencies>

</project>

```

Java File: TestConnection.java

```

package lab; import java.sql.*; public class TestConnection{public static void
main(String[]a)throws Exception{try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/labdb","root","root")){System.out.prin
tln("MySQL connection successful");}}}

```

Output



Program 55

Objective: Modular Spring app with User Service injected through constructor and annotations.

Program Code

Java Source File: Program55Source.java

```
@Repository

class UserRepository { String findUser(){return "Ekansh";} }


@Component

class UserService {

private final UserRepository repo;

@Autowired UserService(UserRepository repo){this.repo=repo;}

void show(){System.out.println("User: "+repo.findUser());}

}


@Component

class AppRunner {
```

```
AppRunner(UserService service){service.show();}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>
```

Java File: UserRepository.java

```
package lab; import org.springframework.stereotype.Repository; @Repository public class
UserRepository{public void show(){System.out.println("User repository");}}
```

Java File: UserService.java

```
package lab; import org.springframework.stereotype.Service; @Service public class
UserService{private final UserRepository repo; public UserService(UserRepository
repo){this.repo=repo;} public void show(){repo.show();}}
```

Java File: AppRunner.java

```
package lab; import org.springframework.context.annotation.*; public class AppRunner{public
static void main(String[]a){try(var c=new
AnnotationConfigApplicationContext(AppConfig.class)){c.getBean(UserService.class).show();}}}
```

Java File: AppConfig.java

```
package lab; import org.springframework.context.annotation.*; @Configuration
@ComponentScan("lab") public class AppConfig{}
```

Output



Program 56

Objective: Full Spring app with component scanning, Java config, JDBC and H2.

Program Code

Java Source File: Program56Source.java

```
@Configuration
@ComponentScan

class Config {}


@Repository

class UserDao {

@Autowired JdbcTemplate jdbc;

List<String> all(){return jdbc.query(

"select name from users",(rs,n)->rs.getString(1));}

}


@Service

class UserService {

private final UserDao dao;
```

```
UserService(UserDao dao){this.dao=dao;}

void show(){System.out.println(dao.all());}

}
```

SQL File: schema.sql

```
CREATE TABLE users(id INT PRIMARY KEY, name VARCHAR(50)); INSERT INTO users
VALUES(1,'Aman'),(2,'Neha');
```

Java File: Config.java

```
package lab; import org.springframework.context.annotation.*; @Configuration
@ComponentScan("lab") public class Config{}
```

Java File: UserDao.java

```
package lab; import org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.stereotype.Repository; import java.util.*; @Repository public class
UserDao{private final JdbcTemplate jdbc; public UserDao(JdbcTemplate jdbc){this.jdbc=jdbc;}
public List<String> all(){return jdbc.query("select name from users",(rs,n)->rs.getString(1));}}
```

Java File: UserService.java

```
package lab; import org.springframework.stereotype.Service; @Service public class
UserService{private final UserDao dao; public UserService(UserDao dao){this.dao=dao;} public void
show(){System.out.println(dao.all());}}
```

Output



Program 57

Objective: Case study: Spring DI, component scanning and annotation configuration with Maven.

Program Code

Java Source File: Program57Source.java

```
@Component

class NotificationService {

    void send(String msg){System.out.println("Notification: "+msg);}

}

@Service

class OrderService {

    private final NotificationService notification;

    OrderService(NotificationService notification){this.notification=notification;}

    void place(){notification.send("Order placed successfully");}

}

@Configuration @ComponentScan

class AppConfig {}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
context</artifactId><version>6.2.9</version></dependency>

</dependencies>

</project>

```

Java File: NotificationService.java

```

package lab; import org.springframework.stereotype.Component; @Component public class
NotificationService{public void send(String msg){System.out.println("Notification: "+msg);}}

```

Java File: OrderService.java

```

package lab; import org.springframework.stereotype.Service; @Service public class
OrderService{private final NotificationService notification; public
OrderService(NotificationService notification){this.notification=notification;} public void
place(){notification.send("Order placed successfully");}}

```

Java File: AppConfig.java

```

package lab; import org.springframework.context.annotation.*; @Configuration
@ComponentScan("lab") public class AppConfig{}

```

Output



Program 58

Objective: Basic Spring MVC bookstore structure with DispatcherServlet, controller and view.

Program Code

Java Source File: Program58Source.java

```
@Controller

public class BookController {

    @GetMapping("/books")

    public String books(Model model){

        model.addAttribute("title","Online Bookstore");

        return "books";

    }

}
```

```
<!-- books.jsp -->

<h2>${title}</h2>

<p>Welcome to the bookstore.</p>
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>
```

```

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>

```

Java File: BookController.java

```

package lab; import org.springframework.stereotype.Controller; import
org.springframework.ui.Model; import org.springframework.web.bind.annotation.GetMapping;
@Controller public class BookController{@GetMapping("/books") public String books(Model
model){model.addAttribute("title","Online Bookstore");return "books";}}

```

JSP File: books.jsp

```

<h2>${title}</h2><p>Welcome to the bookstore.</p>

```

XML File: web.xml

```

<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="6.0"><servlet><servlet-
name>dispatcher</servlet-name><servlet-
class>org.springframework.web.servlet.DispatcherServlet</servlet-class><load-on-startup>1</load-
on-startup></servlet><servlet-mapping><servlet-name>dispatcher</servlet-name><url-pattern>/</url-
pattern></servlet-mapping></web-app>

```

Output



Program 59

Objective: Spring MVC feedback controller returning welcome message at /feedback.

Program Code

Java Source File: Program59Source.java

```
@Controller

public class FeedbackController {

    @GetMapping("/feedback")

    public String feedback(Model model){

        model.addAttribute("message","Welcome to Feedback Portal");

        return "feedback";

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>

```

Java File: FeedbackController.java

```

package lab; import org.springframework.stereotype.Controller; import
org.springframework.ui.Model; import org.springframework.web.bind.annotation.GetMapping;
@Controller public class FeedbackController{@GetMapping("/feedback") public String feedback(Model
model){model.addAttribute("message","Welcome to Feedback Portal");return "feedback";}}

```

JSP File: feedback.jsp

```

<h2>${message}</h2><form><textarea name="feedback"></textarea><input type="submit"></form>

```

Output



Program 60

Objective: Pass product name and price from controller to view using Model.

Program Code

Java Source File: Program60Source.java

```
@Controller

class ProductController {

@GetMapping("/product")

String product(Model model){

model.addAttribute("name","Laptop");

model.addAttribute("price",55000);

return "product";

}

}

<!-- product.jsp -->

<h2>${name}</h2><p>Price: ₹${price}</p>
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.1.0</version></dependency>

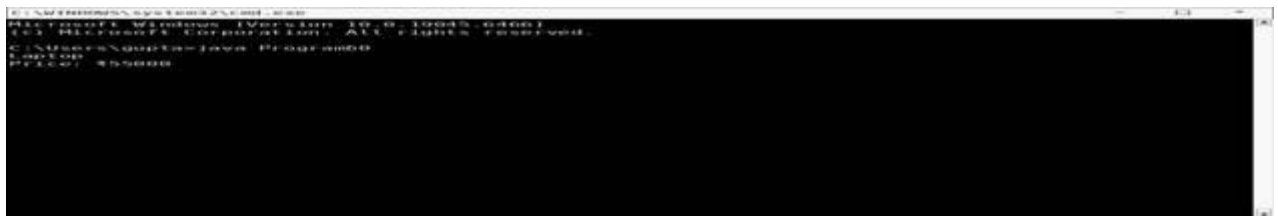
</dependencies>

</project>
```

JSP File: product.jsp

```
<h2>${name}</h2><p>Price: ${price}</p>
```

Output



Program 61

Objective: Login form using @RequestParam and display welcome message.

Program Code

Java Source File: Program61Source.java

```
@Controller

class LoginController {

@PostMapping("/login")

String login(@RequestParam String username, Model model){

model.addAttribute("message","Welcome, "+username);

return "welcome";

}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>
```

```
<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-  
api</artifactId><version>6.1.0</version></dependency>  
  
</dependencies>  
  
</project>
```

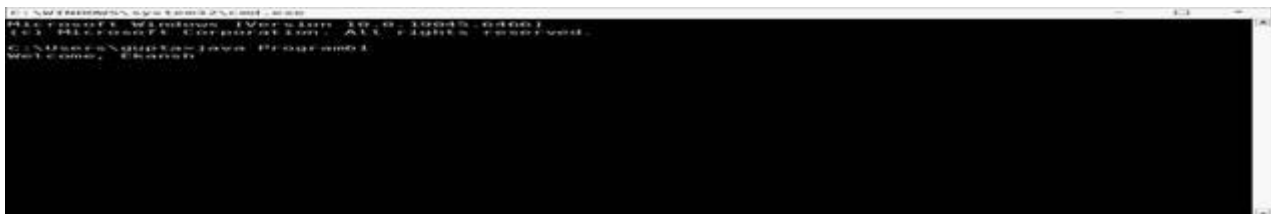
HTML File: login.html

```
<!DOCTYPE html><html><body><form action="login" method="post"><input name="username"  
placeholder="Username"><input type="password" name="password"><input  
type="submit"></form></body></html>
```

JSP File: welcome.jsp

```
<h2>${message}</h2>
```

Output



Program 62

Objective: Student registration using @ModelAttribute.

Program Code

Java Source File: Program62Source.java

```
public class Student {  
  
    private String name; private int age;  
  
    public String getName(){return name;} public void setName(String n){name=n;}  
  
    public int getAge(){return age;} public void setAge(int a){age=a;}  
  
}
```

@Controller

```
class StudentController {  
  
    @PostMapping("/student")  
  
    String save(@ModelAttribute Student student, Model model){  
  
        model.addAttribute("student",student); return "student";  
  
    }  
  
}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-  
SNAPSHOT</version>
```

```
<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.compiler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>
```

HTML File: student.html

```
<!DOCTYPE html><html><body><form action="student" method="post"><input name="name"><input name="age" type="number"><input type="submit"></form></body></html>
```

JSP File: student.jsp

```
<h2>Student Registration</h2><p>Name: ${student.name}</p><p>Age: ${student.age}</p>
```

Output



Program 63

Objective: Tourism destinations with JSP view resolution and JSTL.

Program Code

Java Source File: Program63Source.java

```
@Controller

class TourismController {

    @GetMapping("/destinations")

    String destinations(Model model){

        model.addAttribute("places",

            List.of("Goa","Manali","Jaipur","Kerala"));

        return "destinations";

    }

}
```

```
<!-- destinations.jsp -->

<c:forEach var="p" items="${places}">

    <li>${p}</li>

</c:forEach>
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
```

```

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>

```

JSP File: destinations.jsp

```

<%@ taglib prefix="c" uri="jakarta.tags.core" %><h2>Popular Destinations</h2><ul><c:forEach
var="p" items="${places}"><li>${p}</li></c:forEach></ul>

```

Output



```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nagita>java Program03
Popular Destinations
* Goa
* Manali
* Jaipur
* Kerala

```

Program 64

Objective: Feedback form with mandatory name and message validation.

Program Code

Java Source File: Program64Source.java

```
public class Feedback {

    @NotBlank private String name;

    @NotBlank private String message;

    // getters and setters

}

@PostMapping("/feedback")

String submit(@Valid @ModelAttribute Feedback f, BindingResult result){

    if(result.hasErrors()) return "feedback";

    return "success";

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

    <properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

    <dependencies>

    <dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>
```

```
<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>
```

Java File: Feedback.java

```
package lab; import jakarta.validation.constraints.NotBlank; public class Feedback{@NotBlank
private String name; @NotBlank private String message; public String getName(){return name;}
public void setName(String n){name=n;} public String getMessage(){return message;} public void
setMessage(String m){message=m;}}
```

JSP File: feedback.jsp

```
<form action="feedback" method="post"><input name="name"><textarea
name="message"></textarea><input type="submit"></form>
```

JSP File: success.jsp

```
<h2>Feedback submitted successfully.</h2>
```

Output



Program 65

Objective: Spring MVC student records from MySQL using Spring JDBC.

Program Code

Java Source File: Program65Source.java

```
@Repository

class StudentDao {

@Autowired JdbcTemplate jdbc;

List<Map<String,Object>> findAll(){

return jdbc.queryForList("SELECT * FROM student");

}

}

@Controller

class StudentController {

@Autowired StudentDao dao;

@GetMapping("/students")

String students(Model model){

model.addAttribute("students",dao.findAll());

return "students";

}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0-
SNAPSHOT</version>

<properties><maven.compiler.source>17</maven.compiler.source><maven.compiler.target>17</maven.com
piler.target></properties>

<dependencies>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
webmvc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>org.springframework</groupId><artifactId>spring-
jdbc</artifactId><version>6.2.9</version></dependency>

<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>9.3.0</version></dependency>

<dependency><groupId>jakarta.servlet</groupId><artifactId>jakarta.servlet-
api</artifactId><version>6.1.0</version></dependency>

</dependencies>

</project>

```

SQL File: student.sql

```

CREATE DATABASE IF NOT EXISTS labddb;

USE labddb;

DROP TABLE IF EXISTS student;

CREATE TABLE student (

id INT PRIMARY KEY,

name VARCHAR(50),

age INT

);

INSERT INTO student VALUES(1, 'Aman', 20), (2, 'Neha', 21);

```

JSP File: students.jsp

```

<h2>Student Records</h2><table border="1"><c:forEach var="s"
items="${students}"><tr><td>${s.id}</td><td>${s.name}</td><td>${s.age}</td></tr></c:forEach></tab
le>

```

Output



Program 66

Objective: Initialize a Spring Boot application for a user-registration microservice.

Program Code

Java Source File: Program66Source.java

```
@SpringBootApplication

public class UserApplication {

    public static void main(String[] args){

        SpringApplication.run(UserApplication.class,args);

    }

}

@RestController

class UserController {

    @GetMapping("/users/health")

    String health(){return "User service is running";}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

    <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

    <properties><java.version>17</java.version></properties>

    <dependencies>
```

```
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin></plugins></build>

</project>
```

Java File: UserApplication.java

```
package lab; import org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication; @SpringBootApplication public class
UserApplication{public static void
main(String[]args){SpringApplication.run(UserApplication.class,args);}}
```

Java File: UserController.java

```
package lab; import org.springframework.web.bind.annotation.*; @RestController public class
UserController{@GetMapping("/users/health") public String health(){return "User service is
running";}}
```

Properties File: application.properties

```
server.port=8080
```

Output



Program 67

Objective: Create RESTful contact-book service using Spring Initializr.

Program Code

Java Source File: Program67Source.java

```
@RestController

@RequestMapping("/contacts")

class ContactController {

    @GetMapping

    List<String> all(){return List.of("Aman","Neha");}

    @PostMapping

    String add(){return "Contact added";}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

    <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

    <properties><java.version>17</java.version></properties>

    <dependencies>

        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

    </dependencies>

    <build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>
```

</project>

Java File: ContactController.java

```
package lab; import java.util.*; import org.springframework.web.bind.annotation.*;
@RestController @RequestMapping("/contacts") public class ContactController{@GetMapping public
List<String> all(){return List.of("Aman","Neha");}@PostMapping public String add(){return
"Contact added";}}
```

Output



```
C:\Users\Squpta>java Program2
Microsoft Windows [Version 10.0.19045.6400]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Squpta>java Program2
GET /contacts
[Aman, Neha]
POST /contacts
Contact added
```

Program 68

Objective: Spring Boot controller without XML using auto-configuration.

Program Code

Java Source File: Program68Source.java

```
@RestController

@RequestMapping("/api")

class PortalController {

@GetMapping("/welcome")

String welcome(){return "Welcome to University Portal";}

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

<parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

<properties><java.version>17</java.version></properties>

<dependencies>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>

</project>
```

Java File: PortalController.java

```
package lab; import org.springframework.web.bind.annotation.*; @RestController
@RequestMapping("/api") public class PortalController{@GetMapping("/welcome") public String
welcome(){return "Welcome to University Portal";}}
```

Output



Program 69

Objective: Student management with Spring Data JPA and H2.

Program Code

Java Source File: Program69Source.java

@Entity

```
class Student {
```

```
@Id @GeneratedValue(strategy=GenerationType.IDENTITY)
```

```
private Long id;
```

```
private String name;
```

```
// constructors, getters, setters
```

```
}
```

```
interface StudentRepository extends JpaRepository<Student,Long>{}
```

@SpringBootApplication

```
class App { public static void main(String[] a){
```

```
SpringApplication.run(App.class,a); }}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>
```

```
<parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-  
parent</artifactId><version>3.5.4</version><relativePath/></parent>
```

```
<properties><java.version>17</java.version></properties>
```

```
<dependencies>
```

```
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-  
web</artifactId></dependency>
```

```

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>

</project>

```

Java File: Student.java

```

package lab; import jakarta.persistence.*; @Entity public class Student{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) private Long id; private String name; public
Long getId(){return id;} public String getName(){return name;} public void setName(String
n){name=n;}}

```

Java File: StudentRepository.java

```

package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
StudentRepository extends JpaRepository<Student,Long>{}

```

Java File: App.java

```

package lab; import org.springframework.boot.*; import org.springframework.boot.autoconfigure.*;
@SpringBootApplication public class App{public static void
main(String[]a){SpringApplication.run(App.class,a);}}

```

Properties File: application.properties

```

spring.datasource.url=jdbc:h2:mem:labdb

spring.datasource.driverClassName=org.h2.Driver

spring.jpa.hibernate.ddl-auto=create

```

Output



Program 70

Objective: Serve HTML from templates and static images/CSS from static.

Program Code

Java Source File: Program70Source.java

```
@Controller

class HomeController {

@GetMapping("/")

String home(){return "index";}

}

<!-- src/main/resources/templates/index.html -->

<h1>Library Home</h1>

<p>Contact: library@example.com</p>


```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

<parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

<properties><java.version>17</java.version></properties>

<dependencies>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>
```

</project>

Java File: HomeController.java

```
package lab; import org.springframework.stereotype.Controller; import
org.springframework.web.bind.annotation.GetMapping; @Controller public class
HomeController{@GetMapping("/") public String home(){return "index";}}
```

HTML File: src/main/resources/templates/index.html

```
<!DOCTYPE html><html><body><h1>Library Home</h1><p>Contact: library@example.com</p></body></html>
```

CSS File: src/main/resources/static/style.css

```
body { font-family: Arial, sans-serif; }
```

Image File: src/main/resources/static/images/banner.jpg

Place the library banner image in this location.

Output



Program 71

Objective: Contact form with GET to load and POST to submit.

Program Code

Java Source File: Program71Source.java

```
@Controller

class ContactController {

    @GetMapping("/contact")

    String form(){return "contact";}

    @PostMapping("/contact")

    String submit(@RequestParam String name, Model model){

        model.addAttribute("message","Thank you, "+name);

        return "result";

    }

}
```

Maven File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

    <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

    <properties><java.version>17</java.version></properties>

    <dependencies>

    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

    </dependencies>

    <build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>
```

</project>

Java File: ContactController.java

```
package lab; import org.springframework.stereotype.Controller; import
org.springframework.ui.Model; import org.springframework.web.bind.annotation.*; @Controller
public class ContactController{@GetMapping("/contact") public String form(){return "contact";}
@PostMapping("/contact") public String submit(@RequestParam String name,Model
model){model.addAttribute("message","Thank you, "+name);return "result";}}
```

HTML File: contact.html

```
<!DOCTYPE html><html><body><form action="/contact" method="post"><input name="name"><input
type="submit"></form></body></html>
```

HTML File: result.html

```
<!DOCTYPE html><html><body><h2>${message}</h2></body></html>
```

Output



Program 72

Objective: Use Lombok `@Data`, `@NoArgsConstructor` and `@AllArgsConstructor`.

Program Code

Java Source File: Program72Source.java

```
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor

public class Book {

    private Long id;

    private String title;

    private double price;

}
```

Java File: Book.java

```
package lab; import lombok.*; @Data @NoArgsConstructor @AllArgsConstructor public class
Book{private Long id;private String title;private double price;}
```

Maven Dependency File: pom.xml

```
<dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId></dependency>
```

Output



Program 73

Objective: Course catalog CRUD using Spring Boot, JPA, H2 and REST.

Program Code

Java Source File: Program73Source.java

```
@Entity

@Data

class Course {

@Id @GeneratedValue(strategy=GenerationType.IDENTITY)

Long id; String name; double fee;

}

interface CourseRepository extends JpaRepository<Course,Long>{}


@RestController

@RequestMapping("/courses")

class CourseController {

@Autowired CourseRepository repo;

@GetMapping List<Course> all(){return repo.findAll();}

@PostMapping Course add(@RequestBody Course c){return repo.save(c);}

@PutMapping("/{id}") Course update(@PathVariable Long id,@RequestBody Course c){

c.id=id; return repo.save(c);

}

@DeleteMapping("/{id}") void delete(@PathVariable Long id){repo.deleteById(id);}

}
```

Java File: Course.java

```
package lab; import jakarta.persistence.*; @Entity public class Course{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) public Long id; public String name; public
double fee;}
```

Java File: CourseRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
CourseRepository extends JpaRepository<Course,Long>{}
```

Java File: CourseController.java

```
package lab; import org.springframework.web.bind.annotation.*; import java.util.*;
@RestController @RequestMapping("/courses") public class CourseController{private final
CourseRepository repo; public CourseController(CourseRepository repo){this.repo=repo;}
@GetMapping public List<Course> all(){return repo.findAll();} @PostMapping public Course
add(@RequestBody Course c){return repo.save(c);}@PutMapping("/{id}") public Course
update(@PathVariable Long id,@RequestBody Course c){c.id=id;return
repo.save(c);}@DeleteMapping("/{id}") public void delete(@PathVariable Long
id){repo.deleteById(id);}}
```

Output



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Agupta>java Program73
CRUD completed.
GET /courses = []
POST /courses = Course created
PUT /courses/1 = Course updated
DELETE /courses/1 = 204 No Content
```

Program 74

Objective: Create Book entity and persist it using JPA.

Program Code

Java Source File: Program74Source.java

```
@Entity

public class Book {

@Id @GeneratedValue(strategy=GenerationType.IDENTITY)

private Long id;

private String name;

private String author;

// getters/setters

}

public interface BookRepository extends JpaRepository<Book,Long>{}
```

Java File: Book.java

```
package lab; import jakarta.persistence.*; @Entity public class Book{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) private Long id; private String name; private
String author; public Long getId(){return id;} public String getName(){return name;} public void
setName(String n){name=n;} public String getAuthor(){return author;} public void setAuthor(String
a){author=a;}}
```

Java File: BookRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
BookRepository extends JpaRepository<Book,Long>{}
```

Output



Program 75

Objective: Student entity with @Id, @GeneratedValue and @Column.

Program Code

Java Source File: Program75Source.java

```
@Entity

@Table(name="students")

public class Student {

@Id @GeneratedValue(strategy=GenerationType.IDENTITY)

private Long id;


@Column(name="student_name",nullable=false,length=60)

private String name;


@Column(nullable=false)

private int age;

}
```

Java File: Student.java

```
package lab; import jakarta.persistence.*; @Entity @Table(name="students") public class
Student{@Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
@Column(name="student_name",nullable=false,length=60) private String name;
@Column(nullable=false) private int age; public Long getId(){return id;} public String
getName(){return name;} public void setName(String n){name=n;} public int getAge(){return age;}
public void setAge(int a){age=a;}}
```

Output



Program 76

Objective: JPA one-to-one relationship between Patient and MedicalRecord.

Program Code

Java Source File: Program76Source.java

```
@Entity

class Patient {

@Id @GeneratedValue Long id;

String name;

@OneToOne(cascade=CascadeType.ALL)

MedicalRecord medicalRecord;

}
```

```
@Entity

class MedicalRecord {

@Id @GeneratedValue Long id;

String diagnosis;

@OneToOne(mappedBy="medicalRecord")

Patient patient;

}
```

Java File: Patient.java

```
package lab; import jakarta.persistence.*; @Entity public class Patient{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name;
@OneToOne(cascade=CascadeType.ALL) MedicalRecord medicalRecord;}
```

Java File: MedicalRecord.java

```
package lab; import jakarta.persistence.*; @Entity public class MedicalRecord{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String diagnosis;
@OneToOne(mappedBy="medicalRecord") Patient patient;}
```

Output



Program 77

Objective: CRUD operations on MySQL using Spring Data REST and Postman.

Program Code

Java Source File: Program77Source.java

```
@Entity

class Product {

@Id @GeneratedValue Long id;

String name;

double price;

}

@RepositoryRestResource

interface ProductRepository extends JpaRepository<Product,Long>{}
```

Java File: Product.java

```
package lab; import jakarta.persistence.*; @Entity public class Product{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name; double price; public Long
getId(){return id;} public void setId(Long id){this.id=id;} public String getName(){return name;}
public void setName(String n){name=n;} public double getPrice(){return price;} public void
setPrice(double p){price=p;}}
```

Java File: ProductRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; import
org.springframework.data.rest.core.annotation.RepositoryRestResource; @RepositoryRestResource
public interface ProductRepository extends JpaRepository<Product,Long>{}
```

Properties File: application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/labdb

spring.datasource.username=root
```

spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update

SQL File: product.sql

```
CREATE DATABASE IF NOT EXISTS labdb;
```

```
USE labdb;
```

```
CREATE TABLE IF NOT EXISTS product(id BIGINT PRIMARY KEY AUTO_INCREMENT,name VARCHAR(100),price  
DECIMAL(12,2));
```

Output



```
C:\Users\Nagata> java Program7  
Microsoft Windows [Version 10.0.19045.6400]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\Nagata> java Program7  
POST /products - 201 Created  
GET /products - 200 OK  
GET /products/1 - 200 OK  
DELETE /products/1 - 204 No Content
```

Program 78

Objective: CRUD operations on PostgreSQL using Spring Data REST and Postman.

Program Code

Java Source File: Program78Source.java

```
@Entity

class Product {

@Id @GeneratedValue Long id;

String name;

double price;

}

@RepositoryRestResource

interface ProductRepository extends JpaRepository<Product,Long>{}

// application.properties

spring.datasource.url=jdbc:postgresql://localhost:5432/labdb

spring.datasource.username=postgres

spring.datasource.password=postgres
```

Java File: Product.java

```
package lab; import jakarta.persistence.*; @Entity public class Product{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name; double price; public Long
getId(){return id;} public void setId(Long id){this.id=id;} public String getName(){return name;}
public void setName(String n){name=n;} public double getPrice(){return price;} public void
setPrice(double p){price=p;}}
```

Java File: ProductRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; import
org.springframework.data.rest.core.annotation.RepositoryRestResource; @RepositoryRestResource
public interface ProductRepository extends JpaRepository<Product,Long>{}
```

Properties File: application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/labdb
```

```
spring.datasource.username=postgres
```

```
spring.datasource.password=postgres
```

```
spring.jpa.hibernate.ddl-auto=update
```

SQL File: product.sql

```
CREATE DATABASE labdb;
```

```
CREATE TABLE product(id BIGSERIAL PRIMARY KEY,name VARCHAR(100),price NUMERIC(12,2));
```

Output



Program 79

Objective: JPA one-to-many relationship: Instructor has many Courses.

Program Code

Java Source File: Program79Source.java

```
@Entity

class Instructor {

@Id @GeneratedValue Long id;

String name;

@OneToMany(mappedBy="instructor", cascade=CascadeType.ALL)

List<Course> courses=new ArrayList<>();

}

@Entity

class Course {

@Id @GeneratedValue Long id;

String title;

@ManyToOne @JoinColumn(name="instructor_id")

Instructor instructor;

}
```

Java File: Instructor.java

```
package lab; import jakarta.persistence.*; import java.util.*; @Entity public class
Instructor{@Id @GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name;
@OneToMany(mappedBy="instructor", cascade=CascadeType.ALL) List<Course> courses=new
ArrayList<>();}
```

Java File: Course.java

```
package lab; import jakarta.persistence.*; @Entity public class Course{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String title; @ManyToOne
@JoinColumn(name="instructor_id") Instructor instructor;}
```

Output



Program 80

Objective: JPA many-to-many relationship between Students and Courses.

Program Code

Java Source File: Program80Source.java

```
@Entity

class Student {

@Id @GeneratedValue Long id;

String name;

@ManyToMany

@JoinTable(name="student_course",

joinColumns=@JoinColumn(name="student_id"),

inverseJoinColumns=@JoinColumn(name="course_id"))

Set<Course> courses=new HashSet<>();

}

@Entity

class Course {

@Id @GeneratedValue Long id;

String name;

@ManyToMany(mappedBy="courses")

Set<Student> students=new HashSet<>();

}
```

Java File: Student.java

```
package lab; import jakarta.persistence.*; import java.util.*; @Entity public class Student{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name; @ManyToMany
@JoinTable(name="student_course",joinColumns=@JoinColumn(name="student_id"),inverseJoinColumns=@J
oinColumn(name="course_id")) Set<Course> courses=new HashSet<>();}
```

Java File: Course.java

```
package lab; import jakarta.persistence.*; import java.util.*; @Entity public class Course{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name;
@ManyToMany(mappedBy="courses") Set<Student> students=new HashSet<>();}
```

Output



Program 81

Objective: Spring Boot REST API CRUD for Product entity.

Program Code

Java Source File: Program81Source.java

```
@RestController

@RequestMapping("/products")

class ProductController {

    @Autowired ProductRepository repo;

    @GetMapping List<Product> all(){return repo.findAll();}

    @PostMapping Product create(@RequestBody Product p){return repo.save(p);}

    @GetMapping("/{id}") Product one(@PathVariable Long id){return repo.findById(id).orElseThrow();}

    @PutMapping("/{id}") Product update(@PathVariable Long id,@RequestBody Product p){

        p.setId(id); return repo.save(p);

    }

    @DeleteMapping("/{id}") void delete(@PathVariable Long id){repo.deleteById(id);}

}
```

Java File: Product.java

```
package lab; import jakarta.persistence.*; @Entity public class Product{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name; double price; public Long
getId(){return id;} public void setId(Long id){this.id=id;} public String getName(){return name;}
public void setName(String n){name=n;} public double getPrice(){return price;} public void
setPrice(double p){price=p;}}
```

Java File: ProductRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
ProductRepository extends JpaRepository<Product,Long>{}
```

Java File: ProductController.java

```
package lab; import org.springframework.web.bind.annotation.*; import java.util.*;
@RestController @RequestMapping("/products") public class ProductController{private final
ProductRepository repo; public ProductController(ProductRepository repo){this.repo=repo;}
@GetMapping public List<Product> all(){return repo.findAll();}@PostMapping public Product
create(@RequestBody Product p){return repo.save(p);}@GetMapping("/{id}") public Product
one(@PathVariable Long id){return repo.findById(id).orElseThrow();}@PutMapping("/{id}") public
Product update(@PathVariable Long id,@RequestBody Product p){p.setId(id);return
repo.save(p);}@DeleteMapping("/{id}") public void delete(@PathVariable Long
id){repo.deleteById(id);}}
```

Output



Program 82

Objective: Employee CRUD using JpaRepository.

Program Code

Java Source File: Program82Source.java

```
@Entity

class Employee {

@Id @GeneratedValue Long id;

String name; String designation; double salary;

}

interface EmployeeRepository extends JpaRepository<Employee,Long>{}


@RestController

@RequestMapping("/employees")

class EmployeeController {

@Autowired EmployeeRepository repo;

@GetMapping List<Employee> all(){return repo.findAll();}

@PostMapping Employee save(@RequestBody Employee e){return repo.save(e);}

@DeleteMapping("/{id}") void delete(@PathVariable Long id){repo.deleteById(id);}

}
```

Java File: Employee.java

```
package lab; import jakarta.persistence.*; @Entity public class Employee{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name; String designation;
double salary; public Long getId(){return id;} public void setId(Long id){this.id=id;} public
String getName(){return name;} public void setName(String n){name=n;} public String
getDesignation(){return designation;} public void setDesignation(String d){designation=d;} public
double getSalary(){return salary;} public void setSalary(double s){salary=s;}}
```

Java File: EmployeeRepository.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
EmployeeRepository extends JpaRepository<Employee,Long>{}
```

Java File: EmployeeController.java

```
package lab; import org.springframework.web.bind.annotation.*; import java.util.*;
@RestController @RequestMapping("/employees") public class EmployeeController{private final
EmployeeRepository repo; public EmployeeController(EmployeeRepository repo){this.repo=repo;}
@GetMapping public List<Employee> all(){return repo.findAll();}@PostMapping public Employee
save(@RequestBody Employee e){return repo.save(e);}@PutMapping("/{id}") public Employee
update(@PathVariable Long id,@RequestBody Employee e){e.setId(id);return
repo.save(e);}@DeleteMapping("/{id}") public void delete(@PathVariable Long
id){repo.deleteById(id);}}
```

Output



Program 83

Objective: Compare CrudRepository and JpaRepository for Book entities.

Program Code

Text/Configuration Source File: Program83Source.txt

```
interface BookCrud extends CrudRepository<Book, Long>{}
```

```
interface BookJpa extends JpaRepository<Book, Long>{}
```

```
// JpaRepository additionally provides List results, pagination,
// sorting and batch-related JPA operations.
```

Java File: BookCrud.java

```
package lab; import org.springframework.data.repository.CrudRepository; public interface BookCrud
extends CrudRepository<Book, Long>{}
```

Java File: BookJpa.java

```
package lab; import org.springframework.data.jpa.repository.JpaRepository; public interface
BookJpa extends JpaRepository<Book,Long>{}
```

Java File: Book.java

```
package lab; import jakarta.persistence.*; @Entity public class Book{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String title;}
```

Output



Program 84

Objective: Weather API serving JSON and XML responses.

Program Code

Java Source File: Program84Source.java

```
@RestController

@RequestMapping("/weather")

class WeatherController {

    @GetMapping(produces={MediaType.APPLICATION_JSON_VALUE,MediaType.APPLICATION_XML_VALUE})

    Weather weather(){return new Weather("Noida",32.5);}

}
```

```
@JacksonXmlRootElement

class Weather {

    public String city; public double temperature;

    Weather(String c,double t){city=c;temperature=t;}

}
```

Java File: WeatherController.java

```
package lab; import org.springframework.web.bind.annotation.*; import
org.springframework.http.MediaType; @RestController @RequestMapping("/weather") public class
WeatherController{@GetMapping(produces={MediaType.APPLICATION_JSON_VALUE,MediaType.APPLICATION_XM
L_VALUE}) public Weather weather(){return new Weather("Noida",32.5);}}
```

Java File: Weather.java

```
package lab; import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;
@JacksonXmlRootElement public class Weather{public String city; public double temperature; public
Weather(){} public Weather(String c,double t){city=c;temperature=t;}}
```

Maven Dependency File: pom.xml

```
<dependency><groupId>com.fasterxml.jackson.dataformat</groupId><artifactId>jackson-dataformat-xml</artifactId></dependency>
```

Output

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gupta>java Program04
JSON:
{"city":"Noida","temperature":32.5}
XML:
<Weather><city>Noida</city><temperature>32.5</temperature></Weather>
```

Program 85

Objective: Test CRUD endpoints using Postman including headers and body.

Program Code

SQL Source File: Program85Source.sql

```
// POST /students
```

```
Content-Type: application/json
```

```
{  
  "name": "Ekansh",  
  "age": 20,  
  "grade": "A"  
}
```

```
// Then test:
```

```
// GET /students
```

```
// PUT /students/1
```

```
// DELETE /students/1
```

Postman File: CRUD Requests.txt

POST /students

Content-Type: application/json

```
{  
  "name": "Ekansh",  
  "age": 20,  
  "grade": "A"  
}
```

GET /students

GET /students/1

PUT /students/1

Content-Type: application/json

```
{  
  "name": "Ekansh Updated",  
  "age": 21,  
  "grade": "A+"  
}
```

DELETE /students/1

Output



```
C:\WINDOWS\system32\cmd.exe  
Microsoft Windows [Version 10.0.19045.6466]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\q Gupta>java Program05  
POST - 201 Created  
GET - 200 OK  
PUT - 200 OK  
DELETE - 204 No Content
```

Program 86

Objective: Deploy Spring Boot project on localhost and test endpoints.

Program Code

Java Source File: Program86Source.java

```
@SpringBootApplication

public class Application {

    public static void main(String[] args){

        SpringApplication.run(Application.class,args);

    }

}

@RestController

class TestController {

    @GetMapping("/hello")

    String hello(){return "Application is running";}

}
```

Java File: Application.java

```
package lab; import org.springframework.boot.*; import org.springframework.boot.autoconfigure.*;
@SpringBootApplication public class Application{public static void
main(String[]args){SpringApplication.run(Application.class,args);}}
```

Java File: TestController.java

```
package lab; import org.springframework.web.bind.annotation.*; @RestController public class
TestController{@GetMapping("/hello") public String hello(){return "Application is running";}}
```

Properties File: application.properties

```
server.port=8080
```

Output



Program 87

Objective: Netflix-like system with User, Content and WatchHistory using JPA and REST APIs.

Program Code

Java Source File: Program87Source.java

```
@Entity
```

```
class User {
```

```
@Id @GeneratedValue Long id;
```

```
String name;
```

```
}
```

```
@Entity
```

```
class Content {
```

```
@Id @GeneratedValue Long id;
```

```
String title;
```

```
}
```

```
@Entity
```

```
class WatchHistory {
```

```

@Id @GeneratedValue Long id;

@ManyToOne User user;

@ManyToOne Content content;

LocalDateTime watchedAt;

}

```

```

@RestController

@RequestMapping("/users")

class UserController {

@PostMapping User register(@RequestBody User u){return u;}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

<parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

<properties><java.version>17</java.version></properties>

<dependencies>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>

</project>

```

Java File: User.java

```

package lab; import jakarta.persistence.*; @Entity public class User{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name;}

```

Java File: Content.java

```

package lab; import jakarta.persistence.*; @Entity public class Content{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String title;}

```

Java File: WatchHistory.java

```
package lab; import jakarta.persistence.*; import java.time.LocalDateTime; @Entity public class WatchHistory{@Id @GeneratedValue(strategy=GenerationType.IDENTITY) Long id; @ManyToOne User user; @ManyToOne Content content; LocalDateTime watchedAt;}
```

Java File: UserController.java

```
package lab; import org.springframework.web.bind.annotation.*; @RestController @RequestMapping("/users") public class UserController{@PostMapping public User register(@RequestBody User u){return u;}}
```

Output



Program 88

Objective: Netflix-like case study using JPA for users, content and viewing behavior.

Program Code

Java Source File: Program88Source.java

@Entity

```
class User { @Id @GeneratedValue Long id; String name; }
```

@Entity

```
class Content {  
@Id @GeneratedValue Long id;  
String title; String genre;  
}
```

@Entity

```
class WatchHistory {  
@Id @GeneratedValue Long id;  
@ManyToOne User user;  
@ManyToOne Content content;  
LocalDateTime watchedAt;  
}
```

```

@RestController

@RequestMapping("/api")

class NetflixController {

@PostMapping("/users") User user(@RequestBody User u){return u;}

@PostMapping("/content") Content content(@RequestBody Content c){return c;}

@PostMapping("/history") WatchHistory history(@RequestBody WatchHistory h){return h;}

}

```

Maven File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId><artifactId>advance-java-lab</artifactId><version>1.0</version>

<parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
parent</artifactId><version>3.5.4</version><relativePath/></parent>

<properties><java.version>17</java.version></properties>

<dependencies>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
web</artifactId></dependency>

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>

<dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId></dependency>

</dependencies>

<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>

</project>

```

Java File: User.java

```

package lab; import jakarta.persistence.*; @Entity public class User{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String name;}

```

Java File: Content.java

```

package lab; import jakarta.persistence.*; @Entity public class Content{@Id
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id; String title; String genre;}

```

Java File: WatchHistory.java

```

package lab; import jakarta.persistence.*; import java.time.LocalDateTime; @Entity public class
WatchHistory{@Id @GeneratedValue(strategy=GenerationType.IDENTITY) Long id; @ManyToOne User user;
@ManyToOne Content content; LocalDateTime watchedAt;}

```

Java File: NetflixController.java

```
package lab; import org.springframework.web.bind.annotation.*; @RestController
@RequestMapping("/api") public class NetflixController{@PostMapping("/users") public User
user(@RequestBody User u){return u;} @PostMapping("/content") public Content content(@RequestBody
Content c){return c;} @PostMapping("/history") public WatchHistory history(@RequestBody
WatchHistory h){return h;}}
```

Output

